Machine Learning. Lecture 11: Problems in modern CV

Ivan Solomatin

girafe ai

Harbour.Space BKK

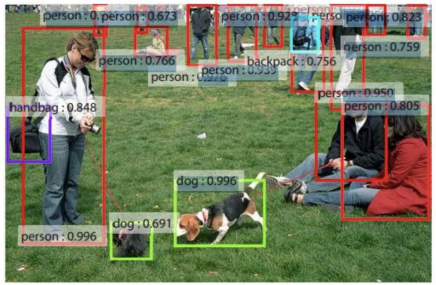
Credits: Vladislav Goncharenko

Outline

- Computer Vision tasks
  - Classification
  - Localisation
  - Detection
  - Semantic Segmentation
  - Instance Segmentation
- Metrics
- Datasets
- R-CNN
- Fast R-CNN
- Faster R-CNN

Computer Vision tasks

Computer Vision tasks



- Classification
- Localisation
- Detection

- Semantic Segmentation
- Instance Segmentation

- Style Transfer
- Adversarial Attacks

- Human Pose Estimation
- Person Re-Identification
- Object Tracking
- ……

Classification

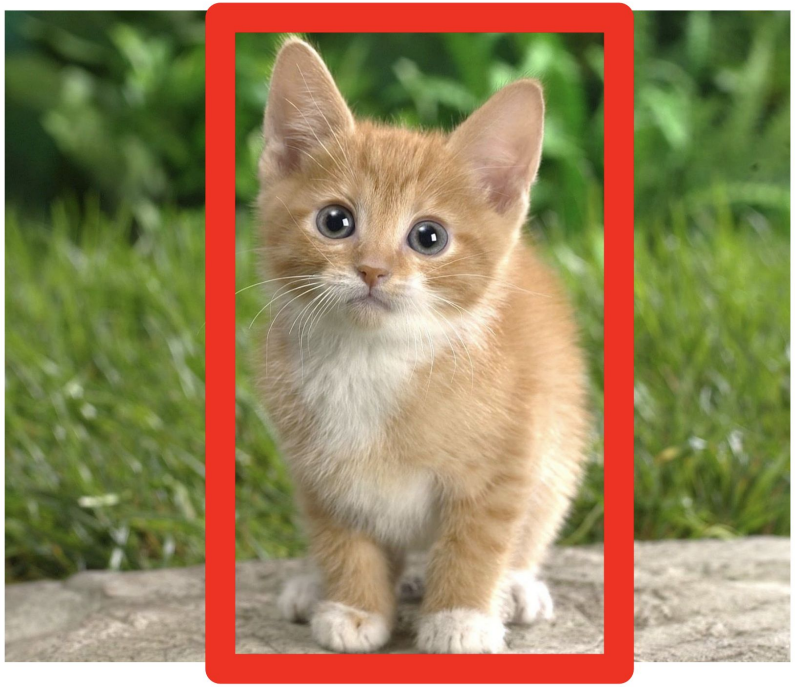
Cat

Input image (C x H x W)

Output

class label (int)

Localisation



Cat

Input image (C x H x W)

Output

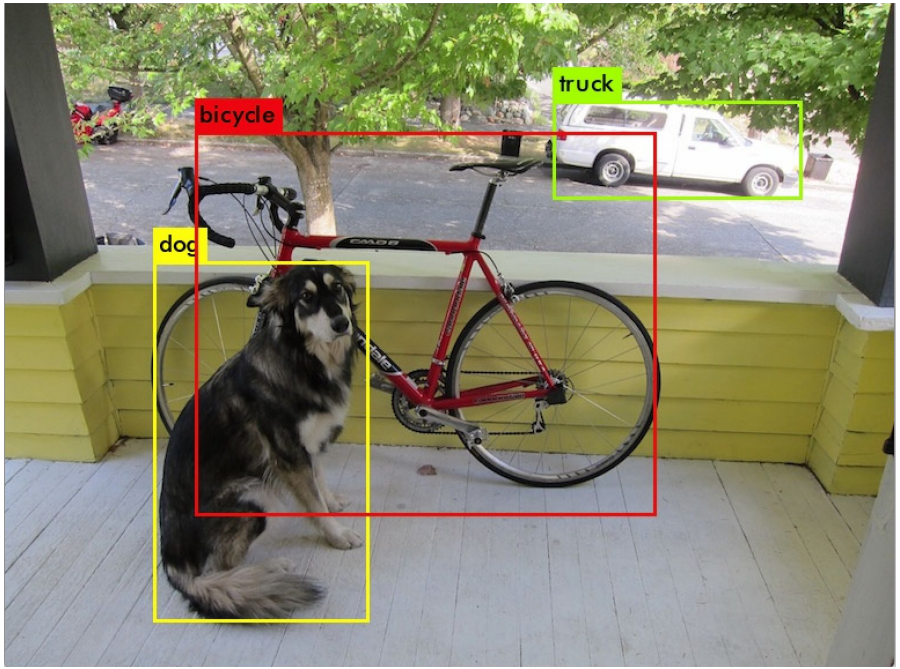
class label (int), BB* coordinates (4 ints)

Assumption

one object per image

* BB - bounding box

Detection



Input image (C x H x W)

Output

class label (int), confidence (0-1 float), BB (4 ints)

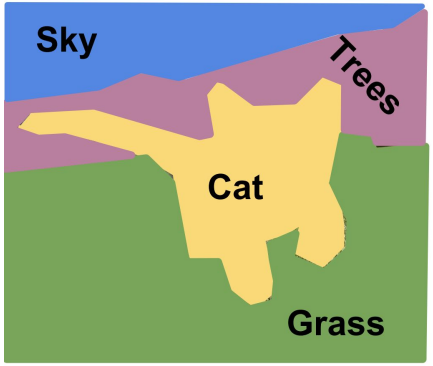
……

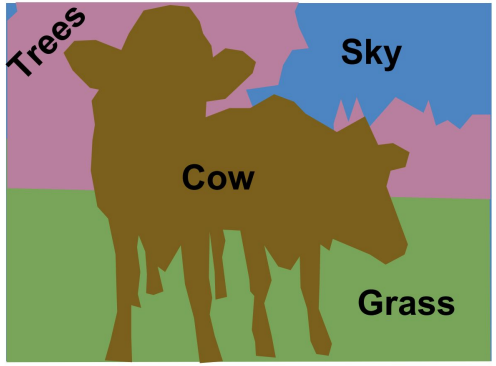
class label (int), confidence (0-1 float), BB (4 ints)

Assumption

multiple objects per image

Semantic Segmentation





Input image (C x H x W)

Output

mask of classes (1 x H x W)

Assumption

just class label for each pixel

Instance Segmentation

Input image (C x H x W)

Output

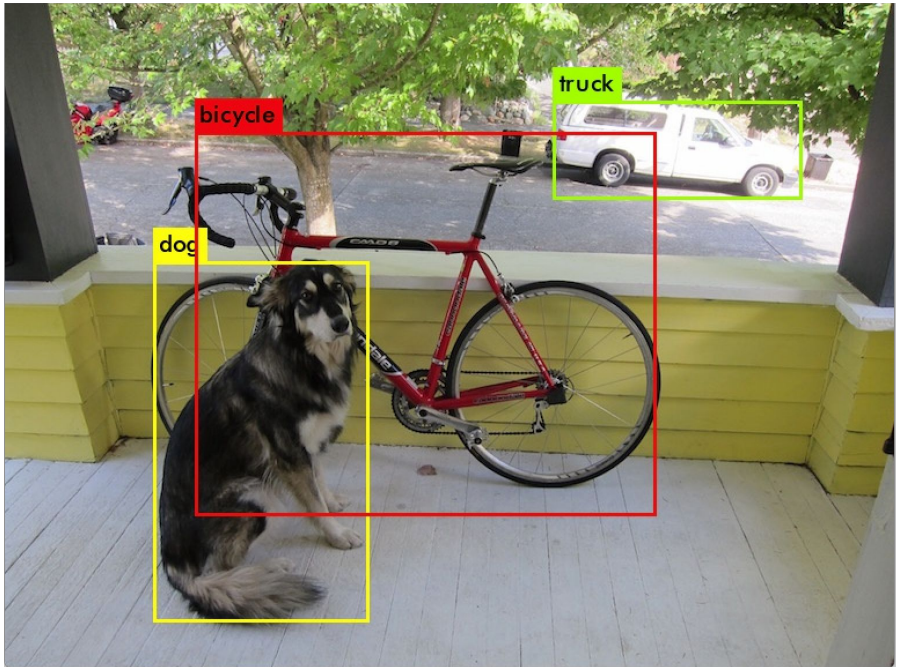
mask of class instances (detection + 1 x H x W)

Assumption

separated masks for each distinct object

Detection metrics

Detection



Input image (C x H x W)

Output

class label (int), confidence (0-1 float), BB (4 ints)

……

class label (int), confidence (0-1 float), BB (4 ints)

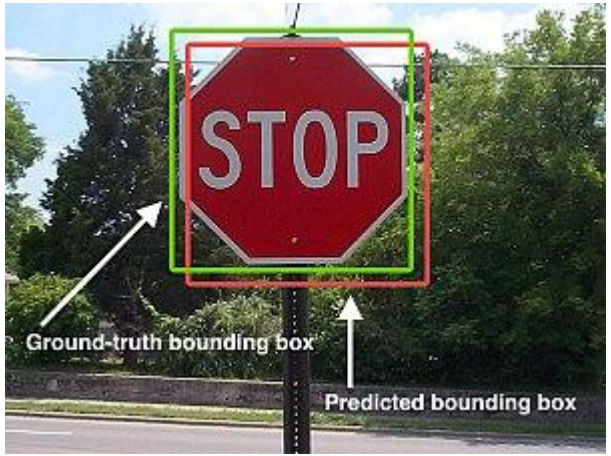
Assumption

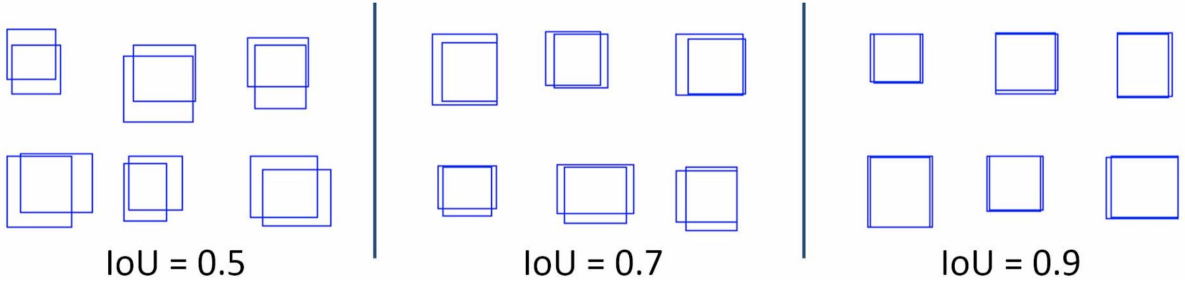
multiple objects per image

Intersection over Union (IoU)

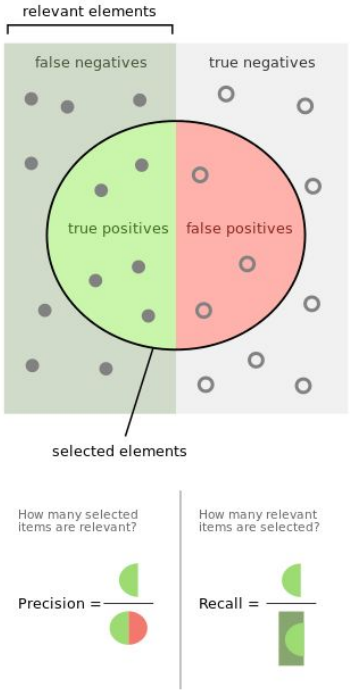
aka Jaccard index

$$IoU = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$





mean Average Precision (mAP)

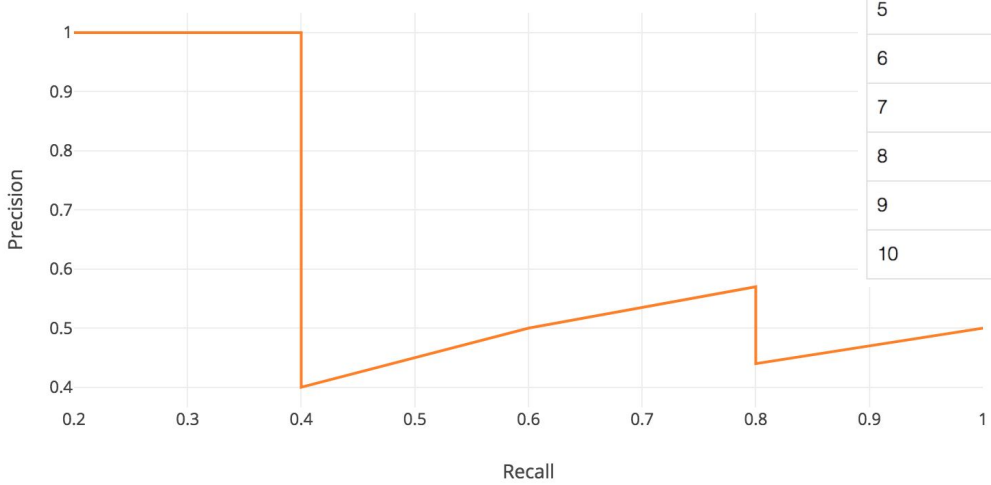


Consider 5 objects and 10 detections

Rank	Correct?	Precision	Recall
1	True	1.0	0.2
2	True	1.0	0.4
3	False	0.67	0.4
4	False	0.5	0.4
5	False	0.4	0.4
6	True	0.5	0.6
7	True	0.57	0.8
8	False	0.5	0.8
9	False	0.44	0.8
10	True	0.5	1.0

mean Average Precision (mAP)

The intention in interpolating the precision/recall curve in this way is to reduce the impact of the “wiggles” in the precision/recall curve, caused by small variations in the ranking of examples.

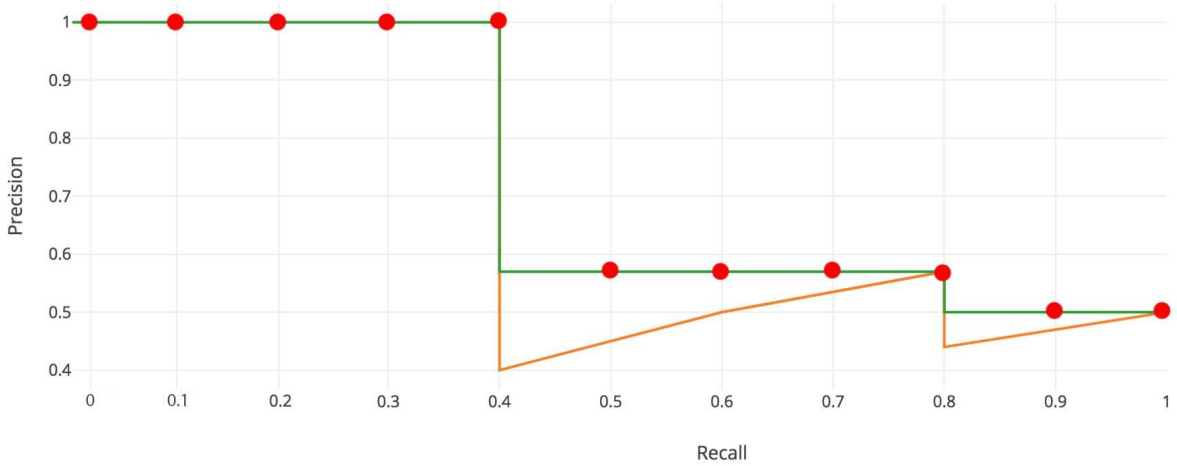


Rank	Correct?	Precision	Recall
1	True	1.0	0.2
2	True	1.0	0.4
3	False	0.67	0.4
4	False	0.5	0.4
5	False	0.4	0.4
6	True	0.5	0.6
7	True	0.57	0.8
8	False	0.5	0.8
9	False	0.44	0.8
10	True	0.5	1.0

mean Average Precision (mAP)

mAP is mean of AP over all classes

$$AP = \frac{1}{11} \times \left(AP_r(0) + AP_r(0.1) + \ldots + AP_r(1.0) \right)$$



COCO mAP

Average Precision (AP):

AP	% AP at IoU=.50:.05:.95 **(primary challenge metric)**
$AP^{IoU=.50}$	% AP at IoU=.50 (PASCAL VOC metric)
$AP^{IoU=.75}$	% AP at IoU=.75 (strict metric)

AP Across Scales:

AP^{small}	% AP for small objects: area < 32^2
AP^{medium}	% AP for medium objects: 32^2 < area < 96^2
AP^{large}	% AP for large objects: area > 96^2

Average Recall (AR):

$AR^{max=1}$	% AR given 1 detection per image
$AR^{max=10}$	% AR given 10 detections per image
$AR^{max=100}$	% AR given 100 detections per image

AR Across Scales:

AR^{small}	% AR for small objects: area < 32^2
AR^{medium}	% AR for medium objects: 32^2 < area < 96^2
AR^{large}	% AR for large objects: area > 96^2

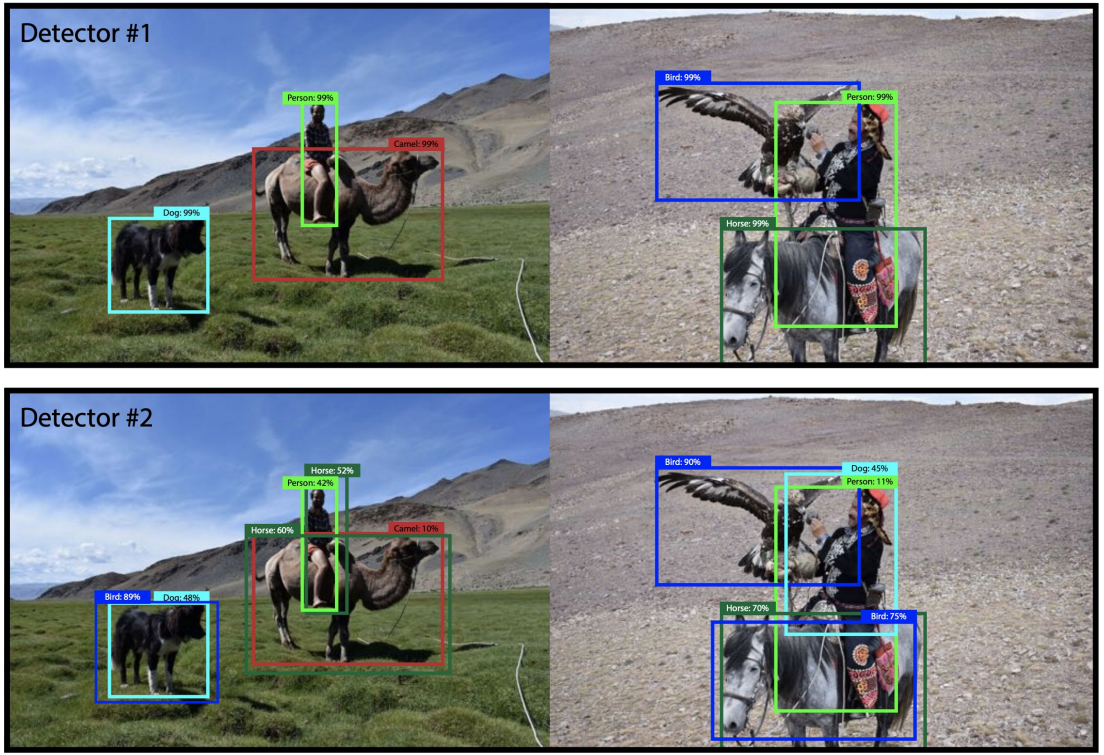
See https://medium.com/@jonathan_hui/map-mean-average-precision-for-object-detection-45c121a31173

mean Average Precision (mAP)

Problems

These two hypothetical detectors are perfect according to mAP over these two images. They are both perfect. Totally equal

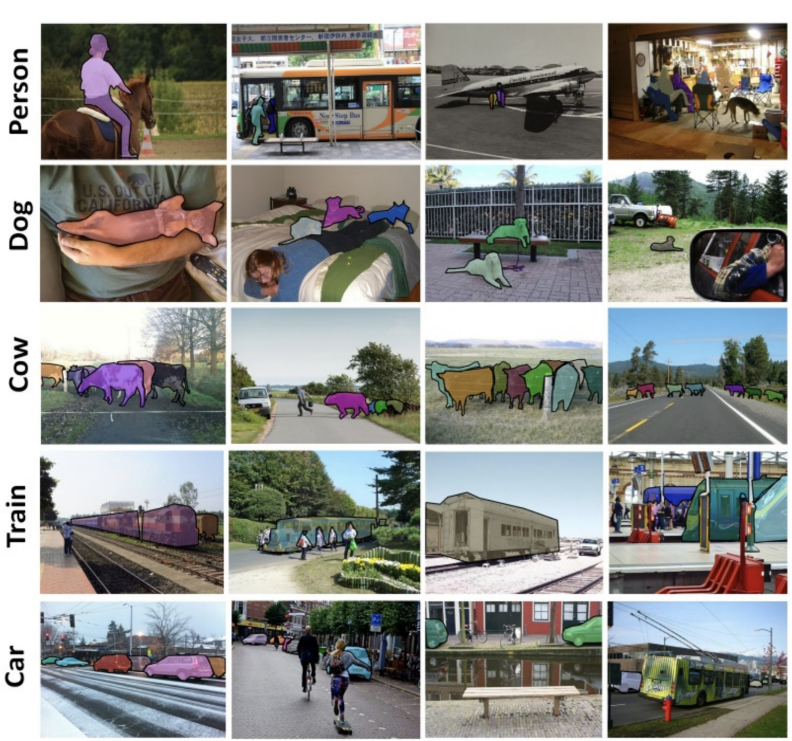
(from YOLOv3 article)

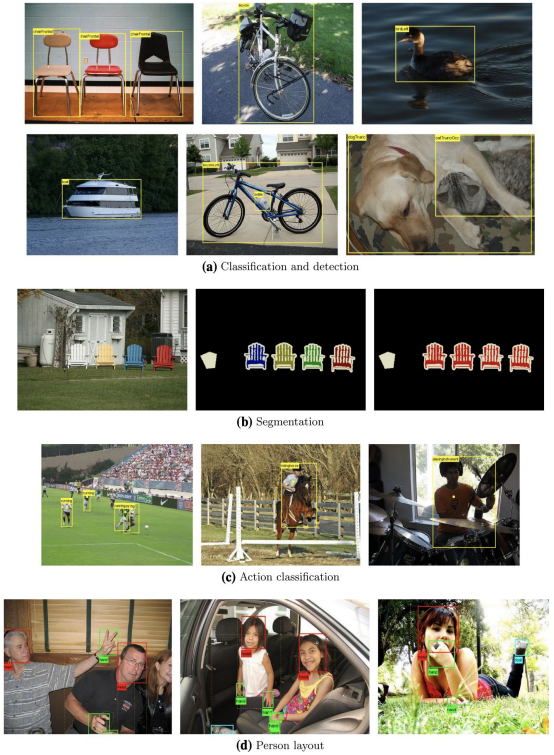


Datasets

Detection datasets

1. PASCAL VOC 2012
2. ImageNet
3. COCO 2017
4. Open Images




(a) Classification and detection

(b) Segmentation

(c) Action classification

(d) Person layout

Thanks to Ilya Zakharkin for materials

PASCAL VOC 2012

Table 1 The VOC classes

Vehicles	Household	Animals	Other
Aeroplane	Bottle	Bird	Person
Bicycle	Chair	Cat	
Boat	Dining table	Cow	
Bus	Potted plant	Dog	
Car	Sofa	Horse	
Motorbike	TV/Monitor	Sheep	
Train			

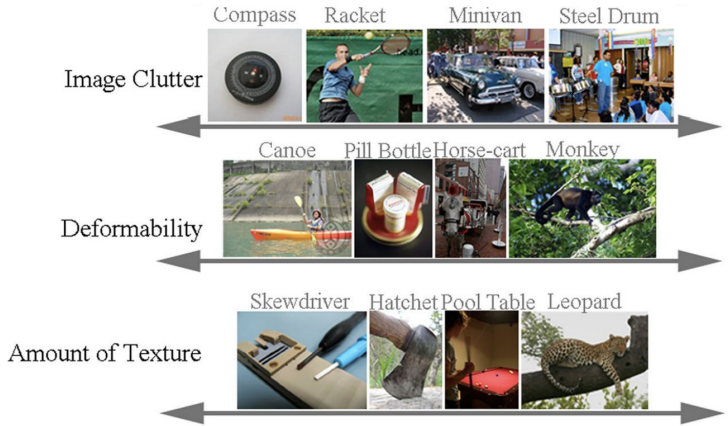
The classes can be considered in a notional taxonomy

Relevant dataset overview:
http://host.robots.ox.ac.uk/pascal/VOC/pubs/everingham15.pdf

- PASCAL Visual Object Classes
- developed since 2005 and 4 classes
- 11,5k images
- 20 classes
- 31,5k detections
- 7k segmentations
- Images came from flickr.com

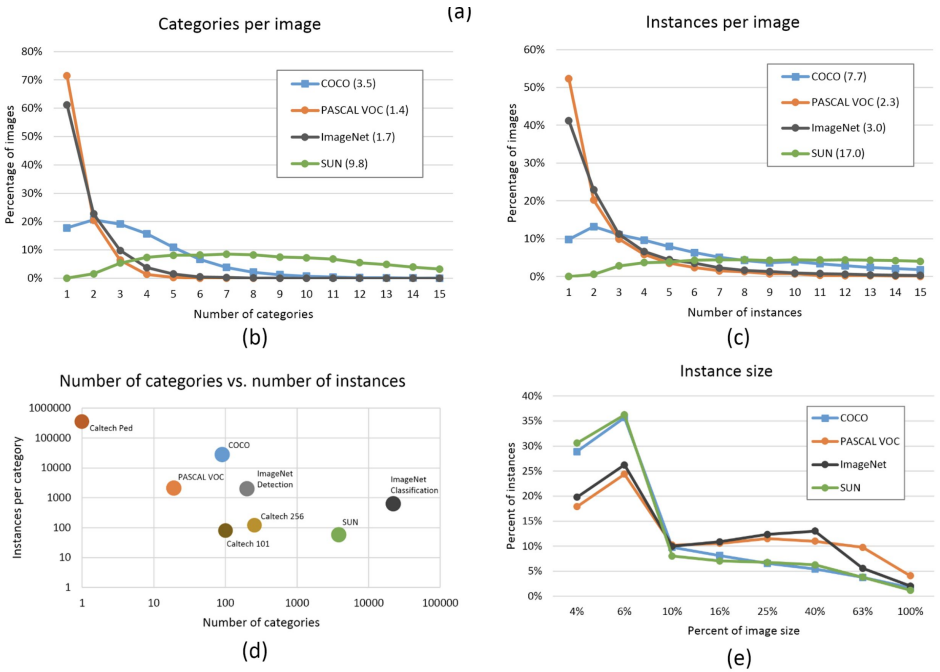
ImageNet

- organized according to the WordNet hierarchy
- 14.2m images
- 1000 classes
- 1m images with BBs
- more fine graded classes than PASCAL
- more diverse objects properties



http://image-net.org/explore

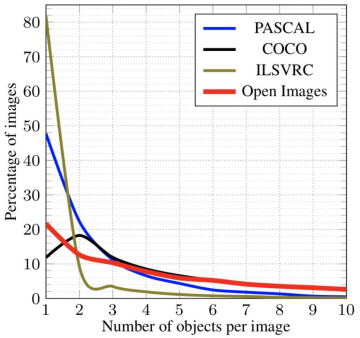
MS COCO 2017

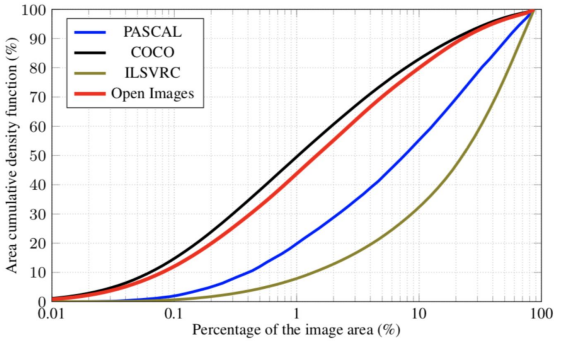


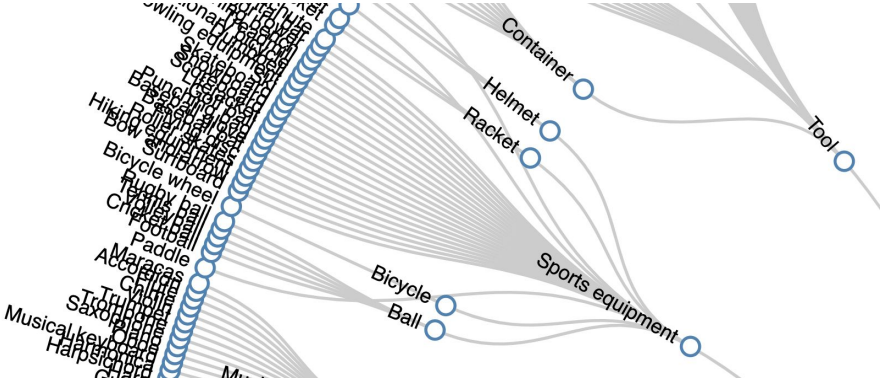
- MicroSoft Common Objects in COntext
- 328k images
- 91 object categories
- 82 having more than 5,000 labeled instances
- 2,5m labeled instances in total
- 250k people with keypoints
- online exploration tool http://cocodataset.org/#explore
- overview https://arxiv.org/pdf/1405.0312.pdf

Open Images

- Provided up to 2020 (v6)
- 15m boxes on 600 categories
- 2,8m instance segmentations on 350 categories
- 36,5m image-level labels on 20k categories
- 391k relationship annotations of 329 relationships
- Extension - 478k crowdsourced images with 6k+ categories







https://storage.googleapis.com/openimages/web/factsfigures.html

Practical notice

Mixup your small datasets with opensourse dataset
to prevent overfitting

R-CNN

Detection as Classification

CAT? NO

DOG? NO

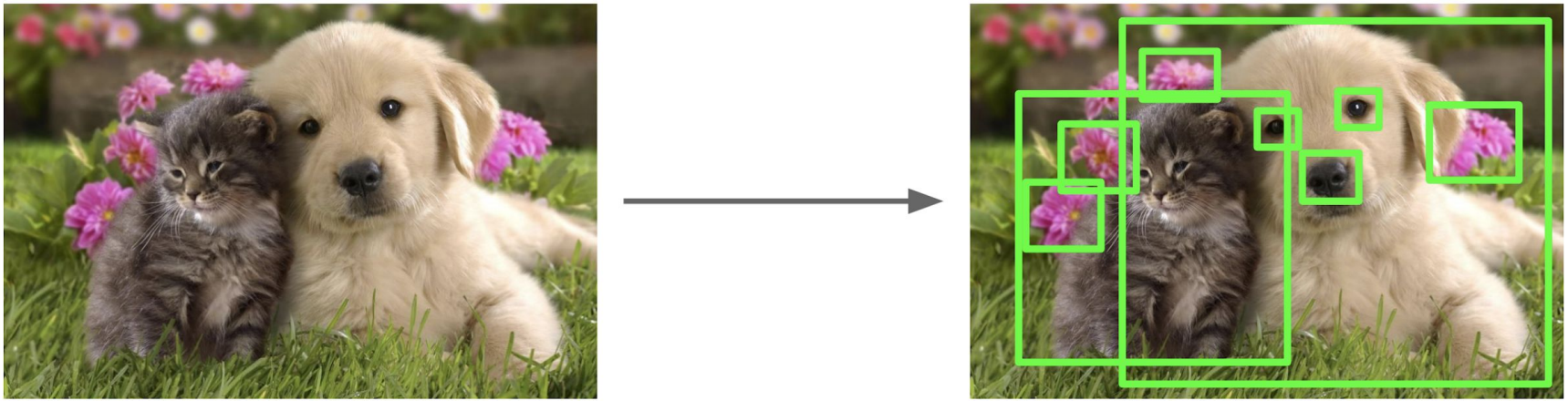
Detection as Classification

CAT? YES!

DOG? NO

Region Proposals

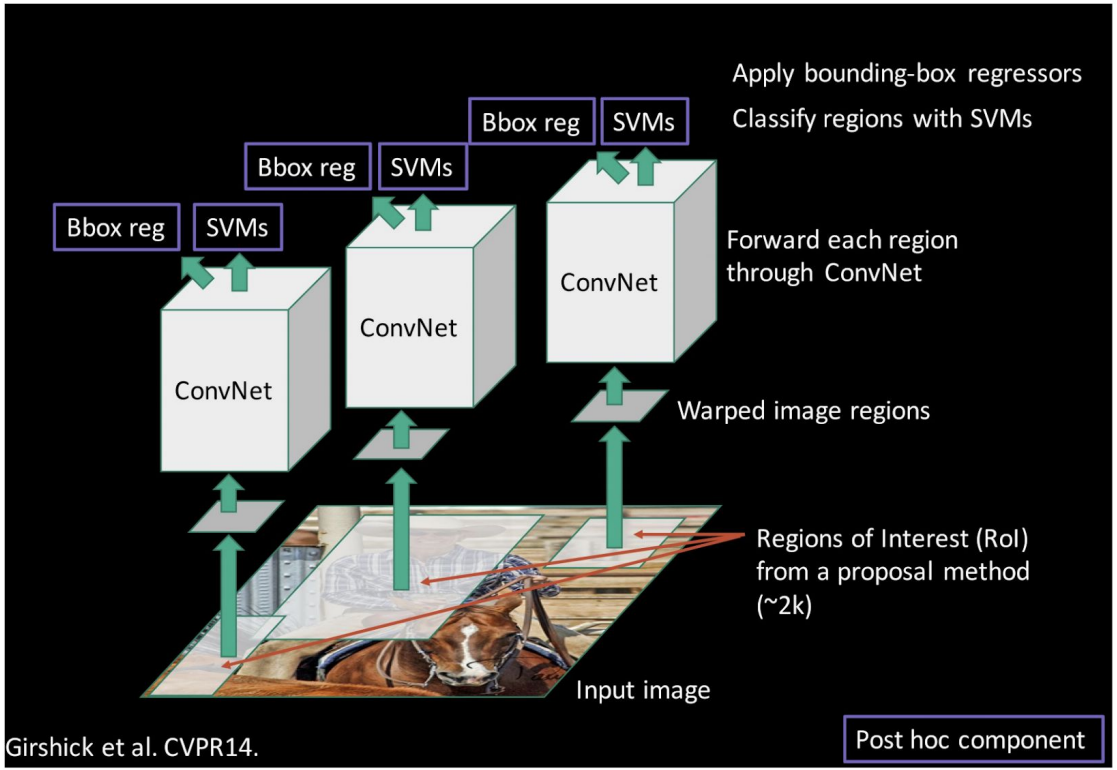
- Find “blobby” image regions that are likely to contain objects
- “Class-agnostic” object detector
- Look for “blob-like” regions



Region Proposals: Many other choices

Method	Approach	Outputs Segments	Outputs Score	Control #proposals	Time (sec.)	Repea-tability	Recall Results	Detection Results
Bing [18]	Window scoring		✓	✓	0.2	⋆⋆⋆	⋆	·
CPMC [19]	Grouping	✓	✓	✓	250	-	⋆⋆	⋆
EdgeBoxes [20]	Window scoring		✓	✓	0.3	⋆⋆	⋆⋆⋆	⋆⋆⋆
Endres [21]	Grouping	✓	✓	✓	100	-	⋆⋆⋆	⋆⋆
Geodesic [22]	Grouping	✓		✓	1	⋆	⋆⋆⋆	⋆⋆
MCG [23]	Grouping	✓	✓	✓	30	⋆	⋆⋆⋆	⋆⋆⋆
Objectness [24]	Window scoring		✓	✓	3	·	⋆	·
Rahtu [25]	Window scoring		✓	✓	3	·	·	⋆
RandomizedPrim's [26]	Grouping	✓		✓	1	⋆	⋆	⋆⋆
Rantalankila [27]	Grouping	✓		✓	10	⋆⋆	·	⋆⋆
Rigor [28]	Grouping	✓		✓	10	⋆	⋆⋆	⋆⋆
SelectiveSearch [29]	Grouping	✓	✓	✓	10	⋆⋆	⋆⋆⋆	⋆⋆⋆
Gaussian				✓	0	·	·	⋆
SlidingWindow				✓	0	⋆⋆⋆	·	·
Superpixels		✓			1	⋆	·	·
Uniform				✓	0	·	·	·

Putting it together: R-CNN

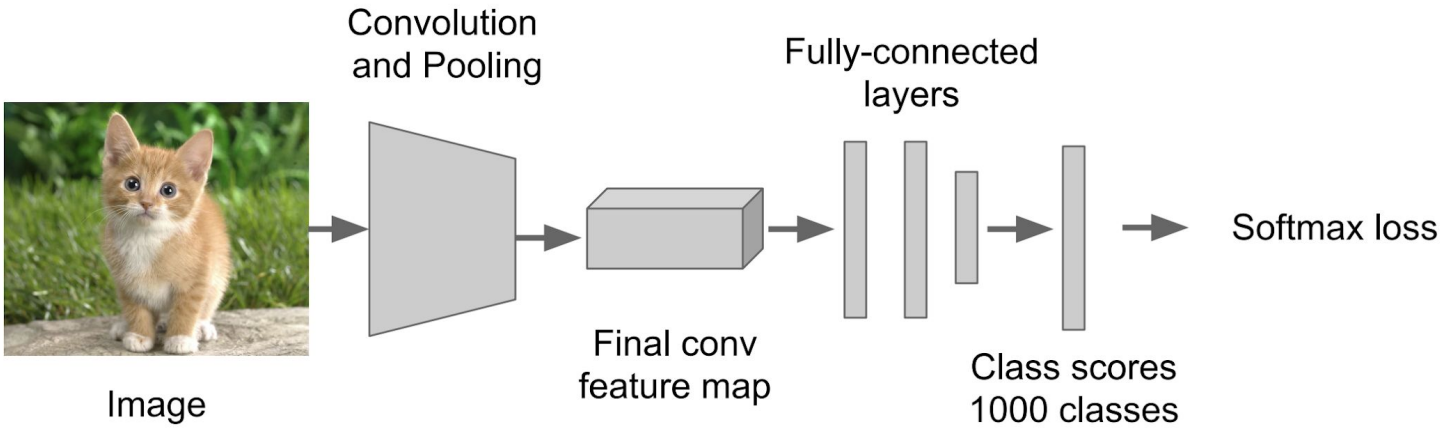


Girschick et al, “Rich feature hierarchies for accurate object detection and semantic segmentation”, CVPR 2014

Slide credit: Ross Girschick

R-CNN Training

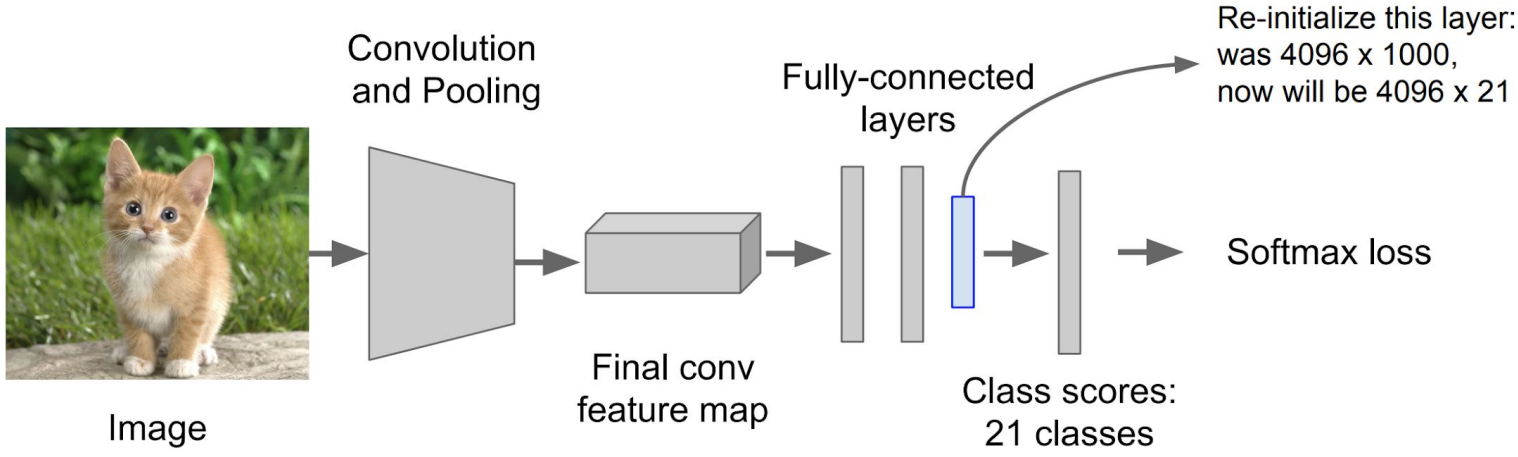
Step 1: Train (or download) a classification model for ImageNet (AlexNet)



R-CNN Training

Step 2: Fine-tune model for detection

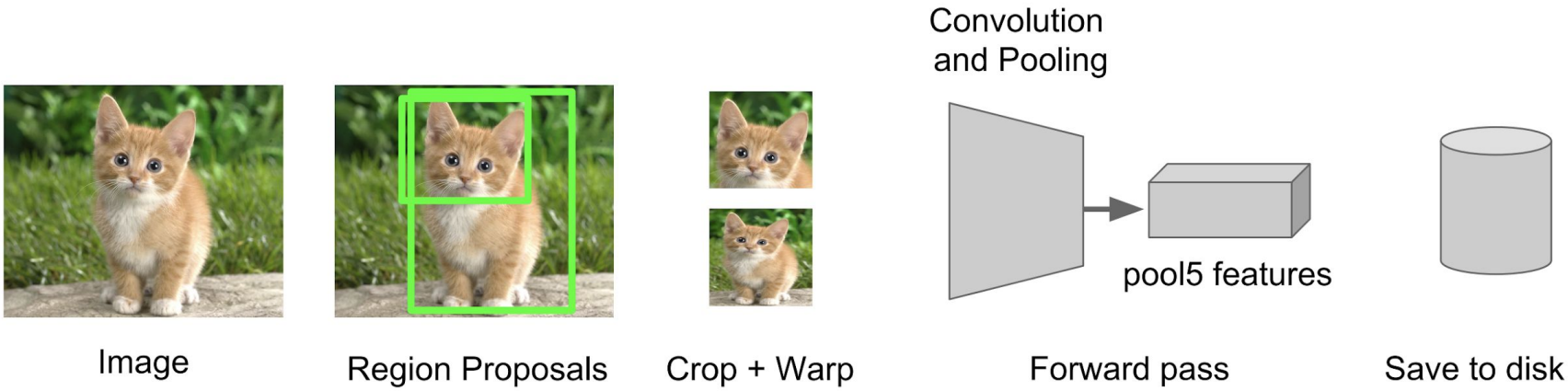
- Instead of 1000 ImageNet classes, want 20 object classes + background
- Throw away final fully-connected layer, reinitialize from scratch
- Keep training model using positive / negative regions from detection images



R-CNN Training

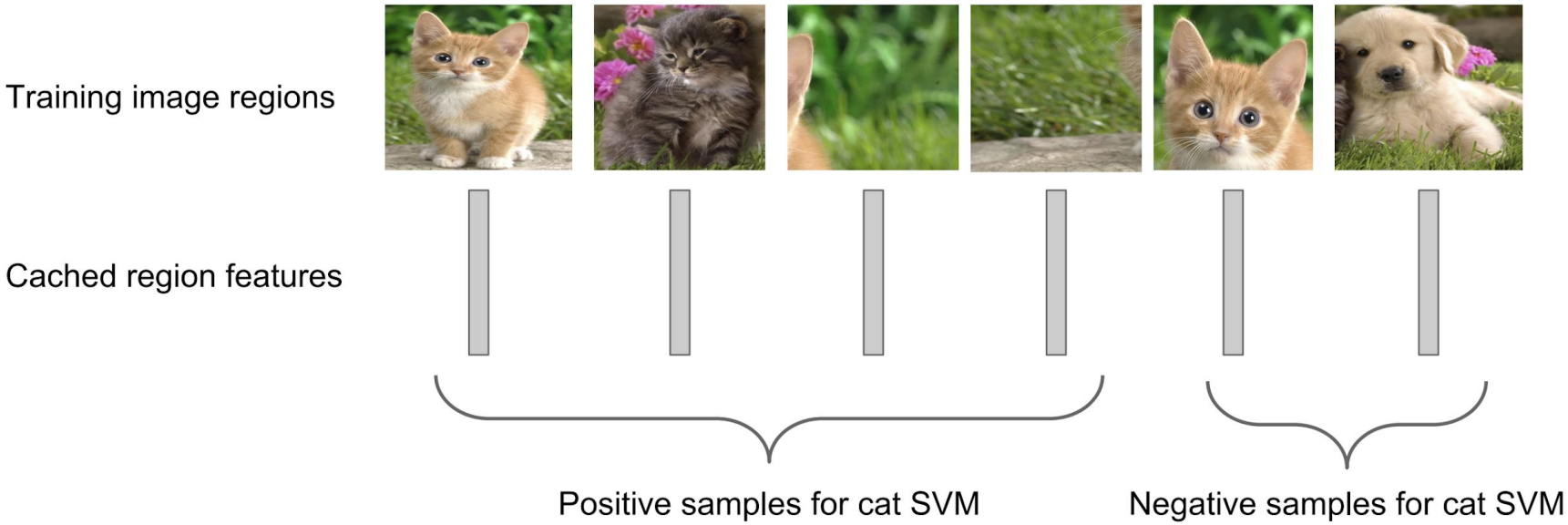
Step 3: Extract features

- Extract region proposals for all images
- For each region: warp to CNN input size, run forward through CNN, save pool5 features to disk
- Have a big hard drive: features are ~200GB for PASCAL dataset!



R-CNN Training

Step 4: Train one binary SVM per class to classify region features



R-CNN Training

Step 5 (bbox regression): For each class, train a linear regression model to map from cached features to offsets to GT boxes to make up for “slightly wrong” proposals

Training image regions

Cached region features

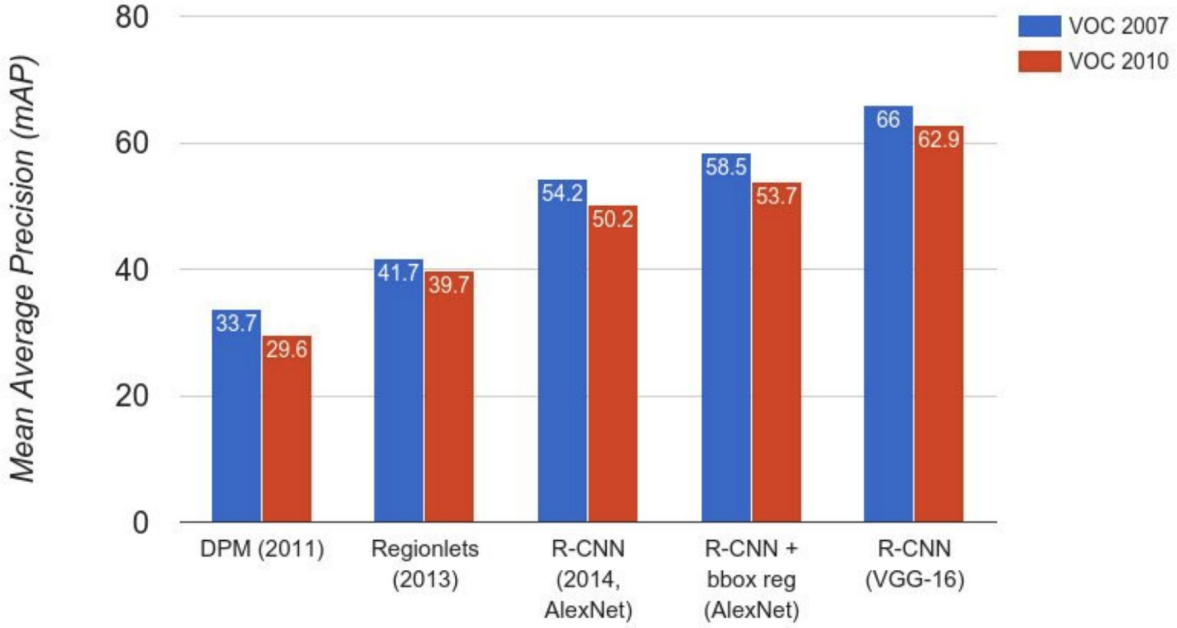
Regression targets
(dx, dy, dw, dh)
Normalized coordinates

(0, 0, 0, 0)
Proposal is good

(.25, 0, 0, 0)
Proposal too far to left

(0, 0, -0.125, 0)
Proposal too wide

R-CNN Results



Wang et al, “Regionlets for Generic Object Detection”, ICCV 2013

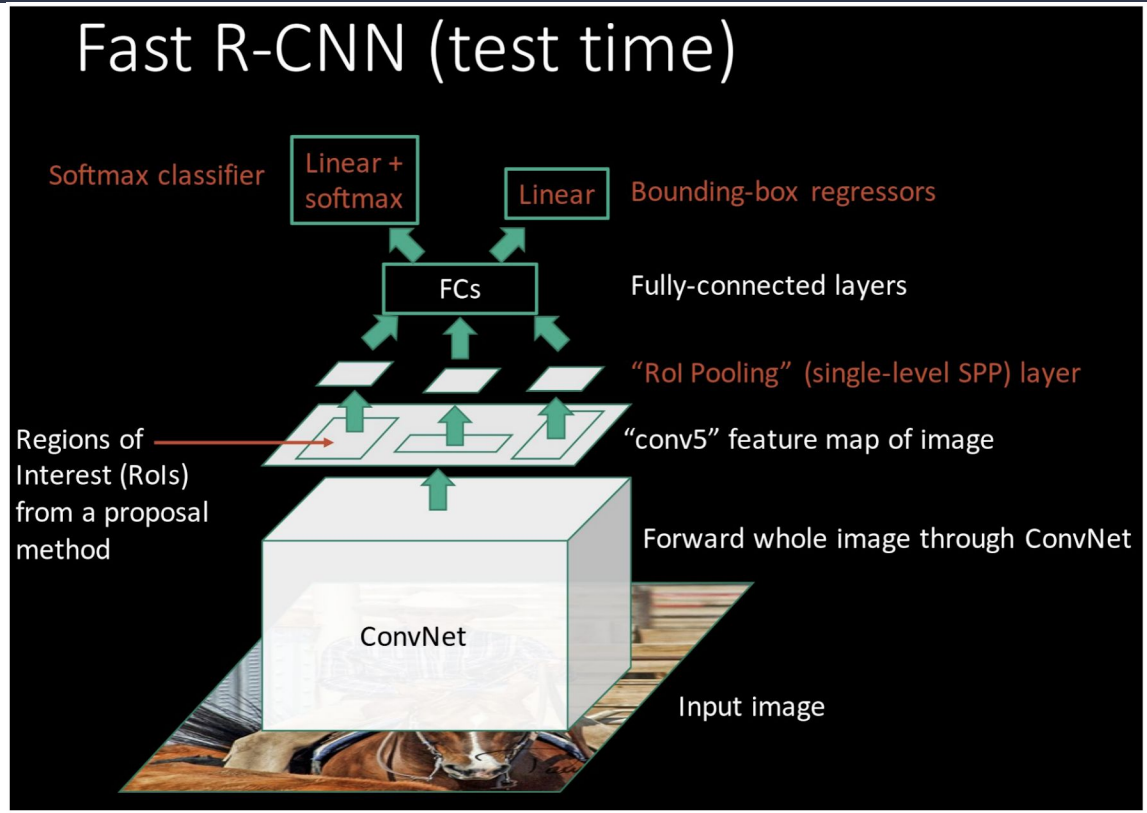
R-CNN Problems

1. Slow at test-time: need to run full forward pass of CNN for each region proposal

2. SVMs and regressors are post-hoc: CNN features not updated in response to SVMs and regressors

3. Complex multistage training pipeline

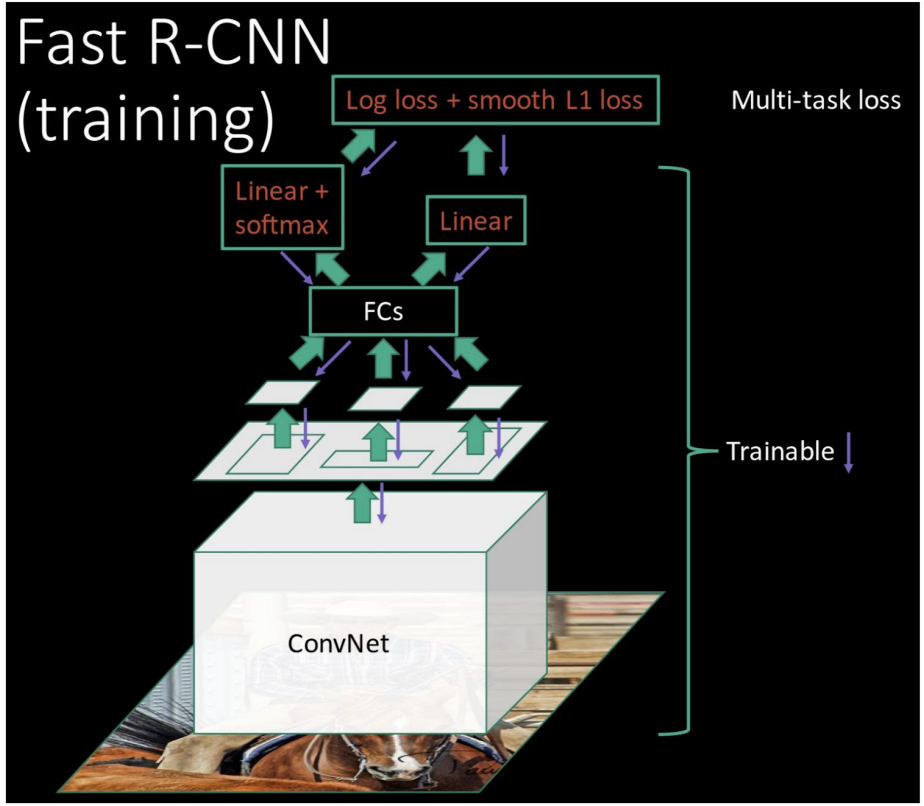
Fast R-CNN



R-CNN Problem #1:
Slow at test-time due to independent forward passes of the CNN

Solution:
Share computation of convolutional layers between proposals for an image

Fast R-CNN



R-CNN Problem #2:
Post-hoc training: CNN not updated in response to final classifiers and regressors

R-CNN Problem #3:
Complex training pipeline

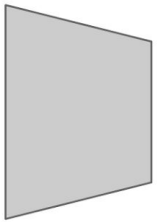
Solution:
Just train the whole system end-to-end all at once!

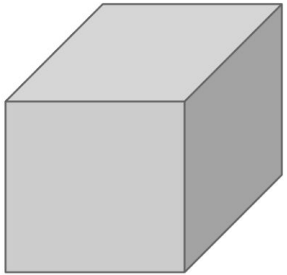
Slide credit: Ross Girschick

Fast R-CNN: Region of Interest Pooling

Hi-res input image:
3 x 800 x 600
with region
proposal

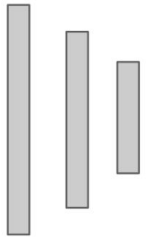
Convolution
and Pooling





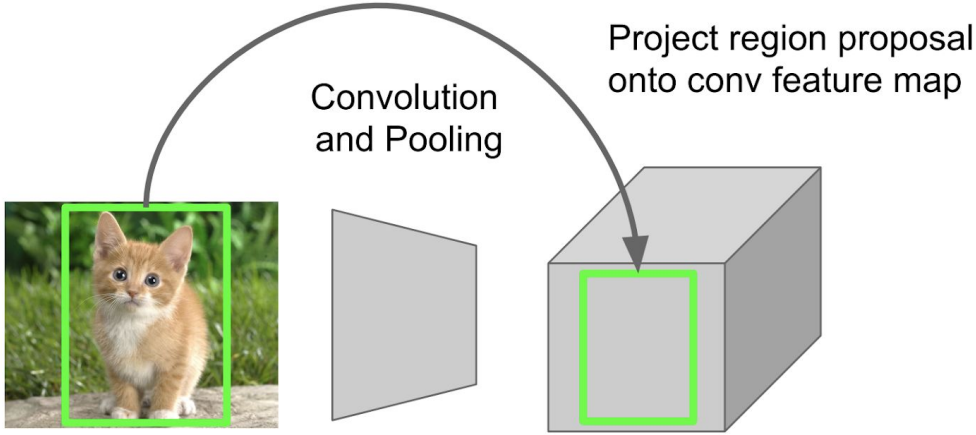
Hi-res conv features:
C x H x W
with region proposal

Fully-connected
layers



Problem: Fully-connected
layers expect low-res conv
features: C x h x w

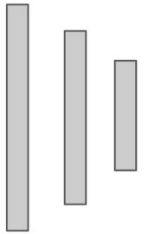
Fast R-CNN: Region of Interest Pooling



Hi-res input image:
3 x 800 x 600
with region
proposal

Hi-res conv features:
C x H x W
with region proposal

Fully-connected
layers

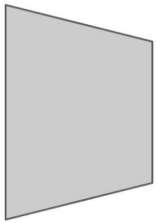


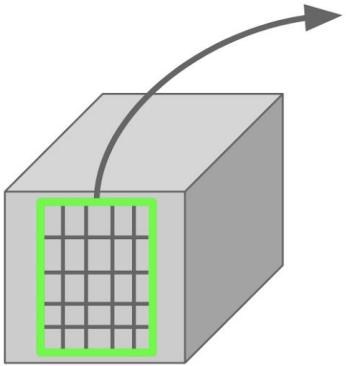
Problem: Fully-connected layers expect low-res conv features: C x h x w

Fast R-CNN: Region of Interest Pooling

Hi-res input image:
3 x 800 x 600
with region
proposal

Convolution
and Pooling

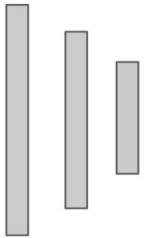




Divide projected
region into h x w grid

Hi-res conv features:
C x H x W
with region proposal

Fully-connected
layers

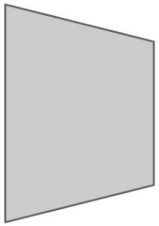


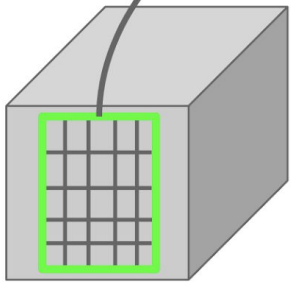
Problem: Fully-connected
layers expect low-res conv
features: C x h x w

Fast R-CNN: Region of Interest Pooling

Hi-res input image:
3 x 800 x 600
with region
proposal

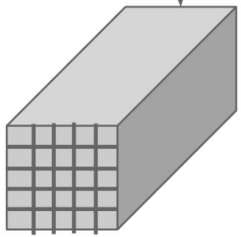
Convolution
and Pooling





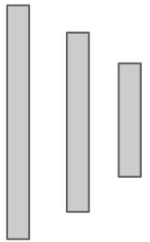
Hi-res conv features:
C x H x W
with region proposal

Max-pool within
each grid cell



RoI conv features:
C x h x w
for region proposal

Fully-connected
layers

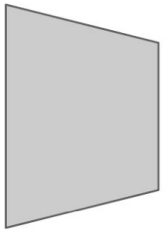


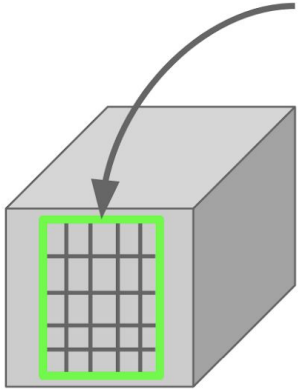
Fully-connected layers expect
low-res conv features:
C x h x w

Fast R-CNN: Region of Interest Pooling

Hi-res input image:
3 x 800 x 600
with region
proposal

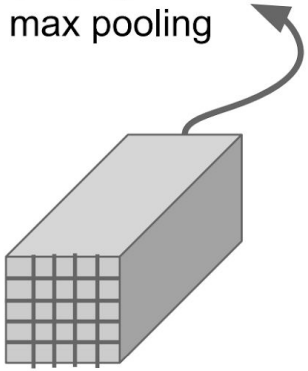
Convolution
and Pooling





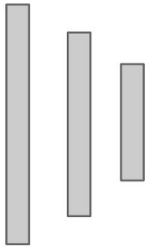
Hi-res conv features:
C x H x W
with region proposal

Can back propagate
similar to max pooling



RoI conv features:
C x h x w
for region proposal

Fully-connected
layers



Fully-connected layers expect
low-res conv features:
C x h x w

Fast R-CNN Results

	R-CNN	Fast R-CNN
Training Time:	84 hours	**9.5 hours**
(Speedup)	1x	**8.8x**
Test time per image	47 seconds	**0.32 seconds**
(Speedup)	1x	**146x**
mAP (VOC 2007)	66.0	**66.9**

Faster!

FASTER!

Better!

Using VGG-16 CNN on Pascal VOC 2007 dataset

Fast R-CNN Problem

Test-time speeds don't include region proposals

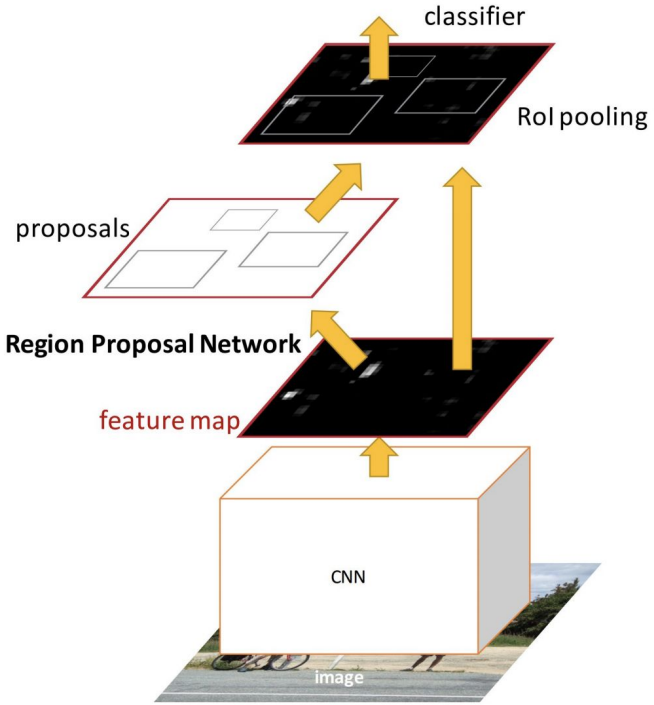
	R-CNN	**Fast R-CNN**
Test time per image	47 seconds	**0.32 seconds**
(Speedup)	1x	**146x**
Test time per image with Selective Search	50 seconds	**2 seconds**
(Speedup)	1x	**25x**

Fast R-CNN Problem Solution

Test-time speeds don't include region proposals
Just make the CNN do region proposals too!

	R-CNN	**Fast R-CNN**
Test time per image	47 seconds	**0.32 seconds**
(Speedup)	1x	**146x**
Test time per image with Selective Search	50 seconds	**2 seconds**
(Speedup)	1x	**25x**

Faster R-CNN



Insert a **Region Proposal Network (RPN)** after the last convolutional layer

RPN trained to produce region proposals directly; no need for external region proposals!

After RPN, use RoI Pooling and an upstream classifier and bbox regressor just like Fast R-CNN

Ren et al, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks", NIPS 2015

Slide credit: Ross Girschick

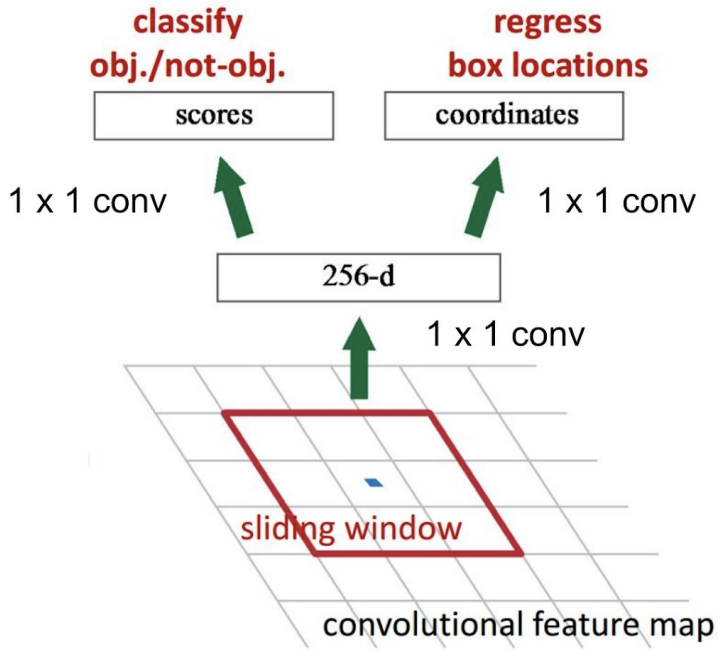
Faster R-CNN: Region Proposal Network

Slide a small window on the feature map

Build a small network for:
- classifying object or not-object, and
- regressing bbox locations

Position of the sliding window provides localization information with reference to the image

Box regression provides finer localization information with reference to this sliding window



Slide credit: Kaiming He

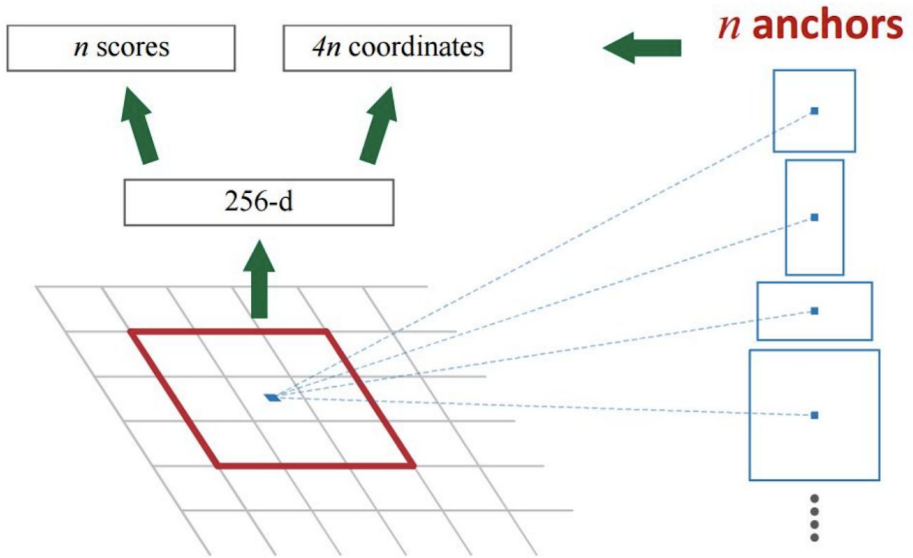
Faster R-CNN: Region Proposal Network

Use **N anchor boxes** at each location

Anchors are **translation invariant**: use the same ones at every location

Regression gives offsets from anchor boxes

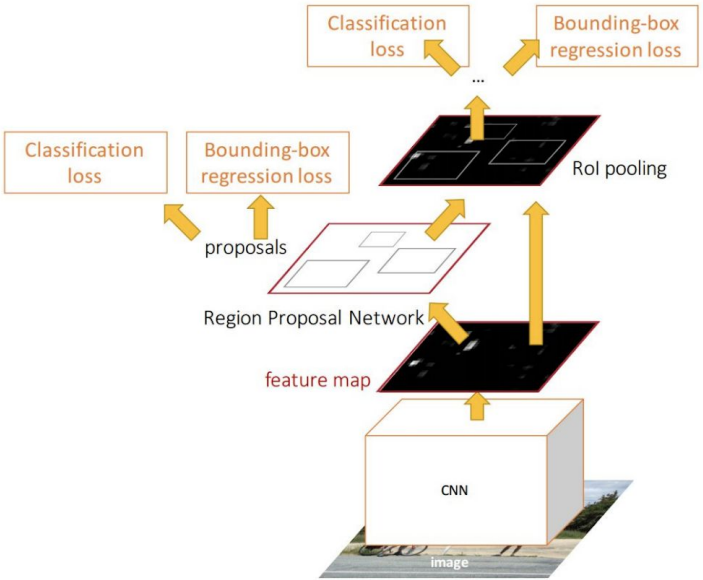
Classification gives the probability that each (regressed) anchor shows an object



Faster R-CNN: Training

In the paper: Ugly pipeline
- Use alternating optimization to train RPN, then Fast R-CNN with RPN proposals, etc.
- More complex than it has to be

Since publication: Joint training!
One network, four losses
- RPN classification (anchor good / bad)
- RPN regression (anchor -> proposal)
- Fast R-CNN classification (over classes)
- Fast R-CNN regression (proposal -> box)

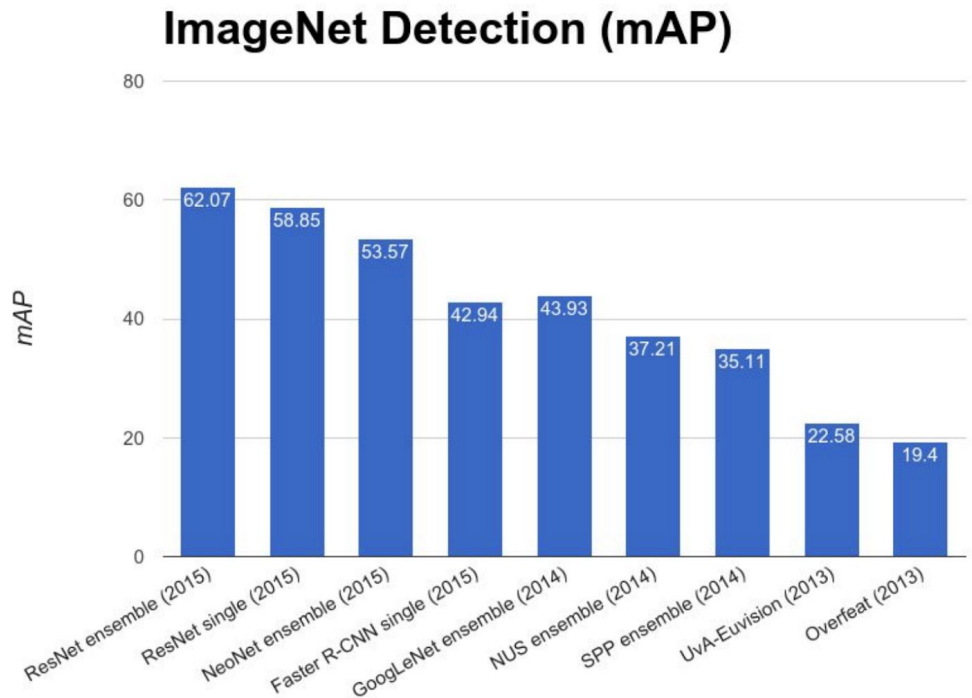


Slide credit: Ross Girschick

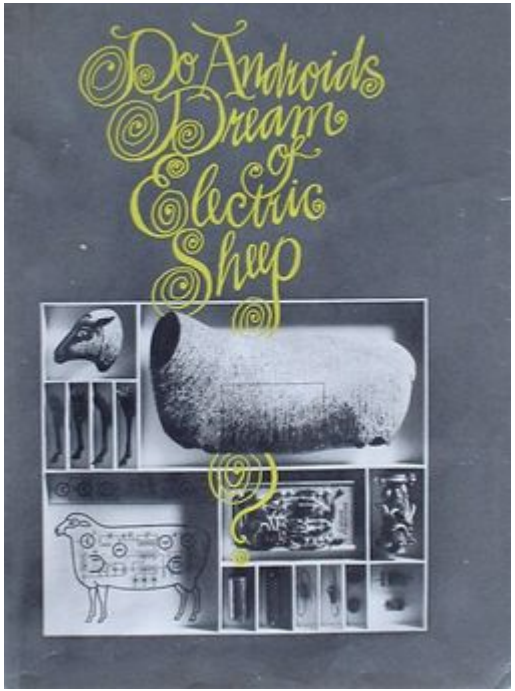
Faster R-CNN: Results

	R-CNN	**Fast R-CNN**	**Faster R-CNN**
Test time per image (with proposals)	50 seconds	2 seconds	**0.2 seconds**
(Speedup)	1x	25x	**250x**
mAP (VOC 2007)	66.0	**66.9**	**66.9**

ImageNet Detection 2013 - 2015



More CV to come...



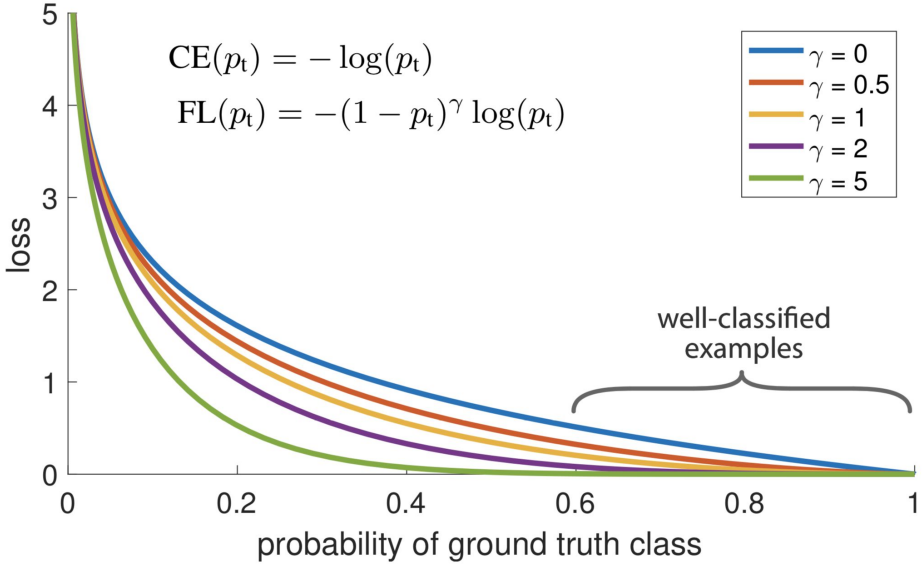
Next time will:

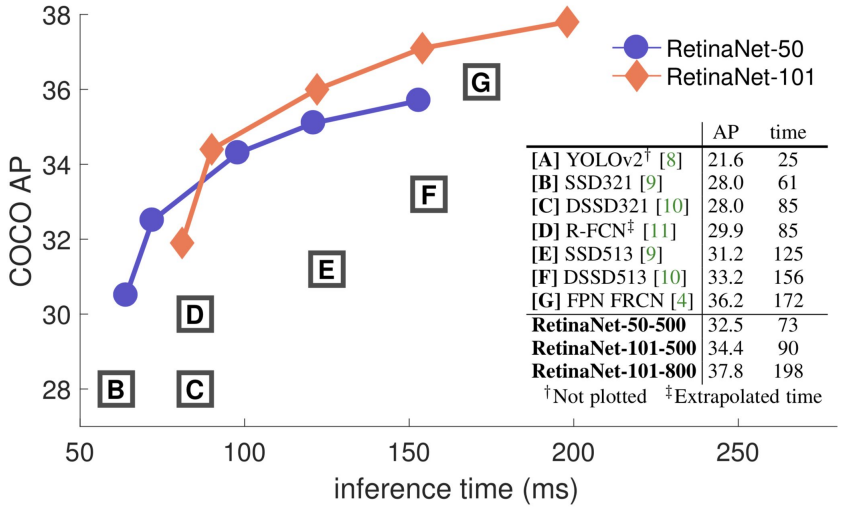
- try to Only Look Once at an image
- know Do Androids Dream of Electric Sheep?

Outline

- Focal Loss
- Non-Maximum Suppression
- YOLO
  - v1
  - v2
  - V3
- Separable convolutions
- MobileNet

Focal Loss



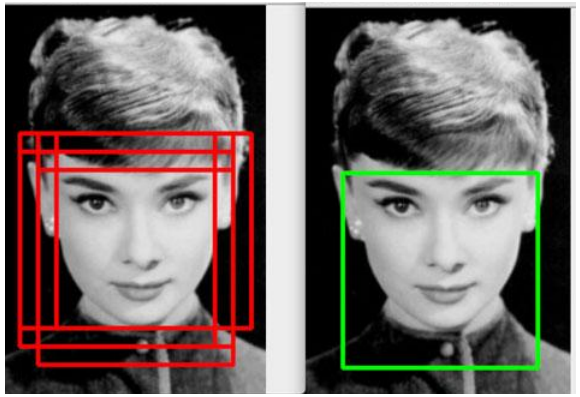


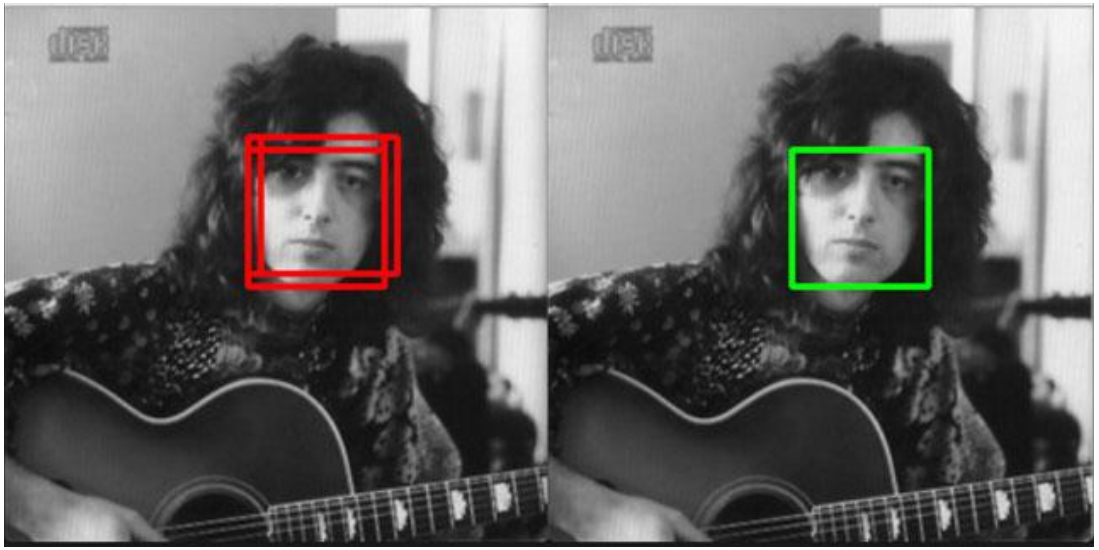
	AP	time
[A] YOLOv2† [8]	21.6	25
[B] SSD321 [9]	28.0	61
[C] DSSD321 [10]	28.0	85
[D] R-FCN‡ [11]	29.9	85
[E] SSD513 [9]	31.2	125
[F] DSSD513 [10]	33.2	156
[G] FPN FRCN [4]	36.2	172
RetinaNet-50-500	32.5	73
RetinaNet-101-500	34.4	90
RetinaNet-101-800	37.8	198

†Not plotted ‡Extrapolated time

Video talk

Non-Maximum Suppression





You Only Look Once (YOLO)

YOLOv1-v2

Original Joseph Redmon slides from conferences

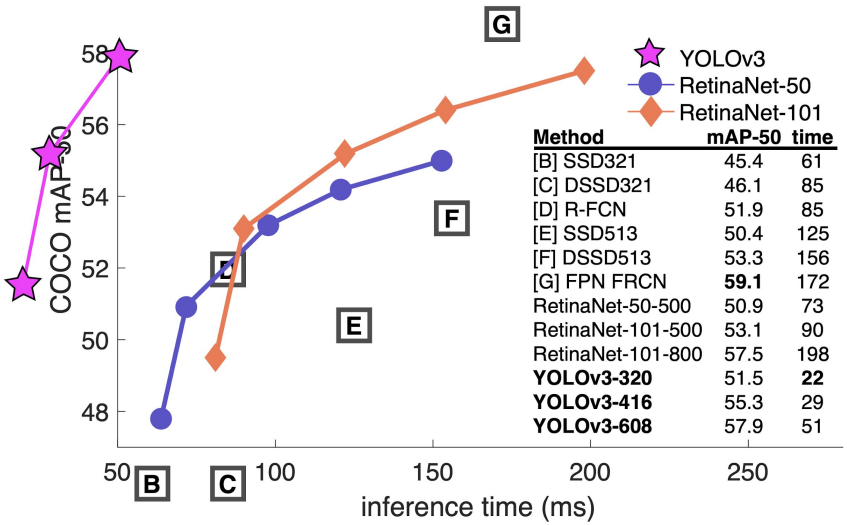
- YOLOv1
- YOLOv2

Also CVPR 2016 talk video

YOLOv3

- Bounding Box Prediction
- Class Prediction
- Predictions Across Scales
- Feature Extractor

All articles published on [Joseph's website]

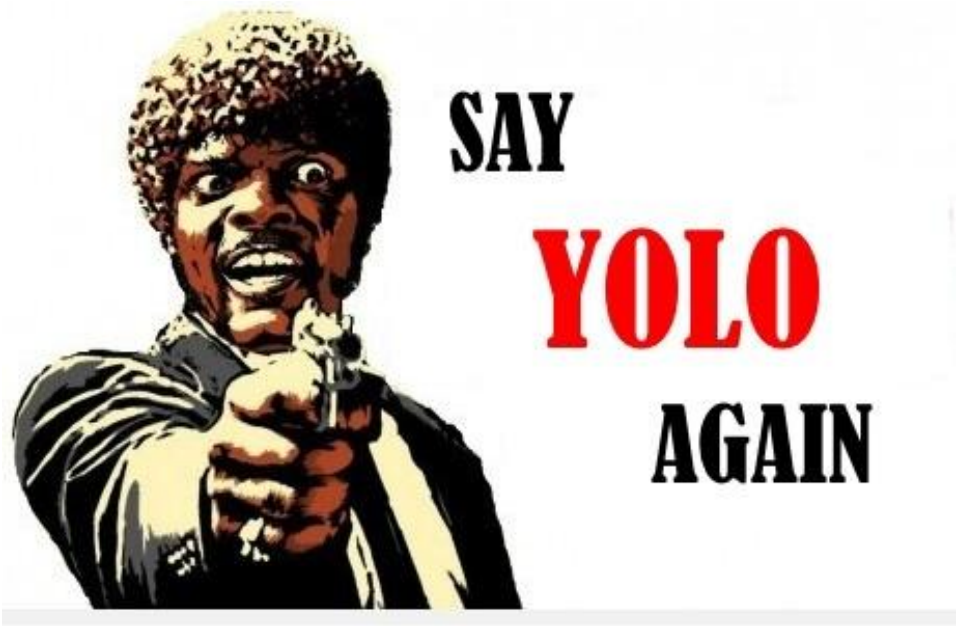


Practical notes

Write your own YOLO from scratch
https://blog.paperspace.com/how-to-implement-a-yolo-object-detector-in-pytorch/

Original YOLO written on Darknet - custom NN framework

Weights import to TF via
https://github.com/thtrieu/darkflow



Current state

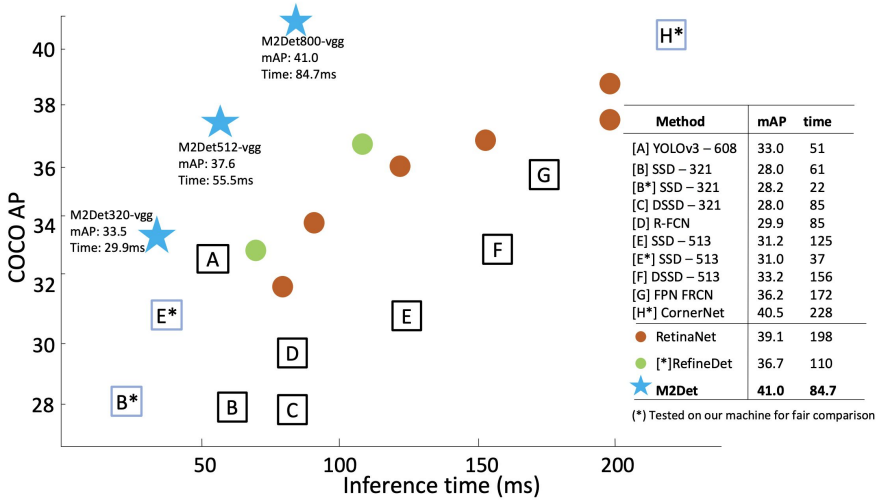


Figure 5: Speed (ms) vs. accuracy (mAP) on COCO *test-dev*.

M2Det: A Single-Shot Object Detector based on Multi-Level Feature Pyramid Network

https://arxiv.org/pdf/1811.04533.pdf

Who is responsible for all these YOLOs

https://docs.google.com/spreadsheets/d/1BxI0SkyFGNvfzLLM_46xZ3i5crTst7jsUKKuZZDWrA0/edit#gid=0

Tools for detection

https://github.com/roboflow/supervision

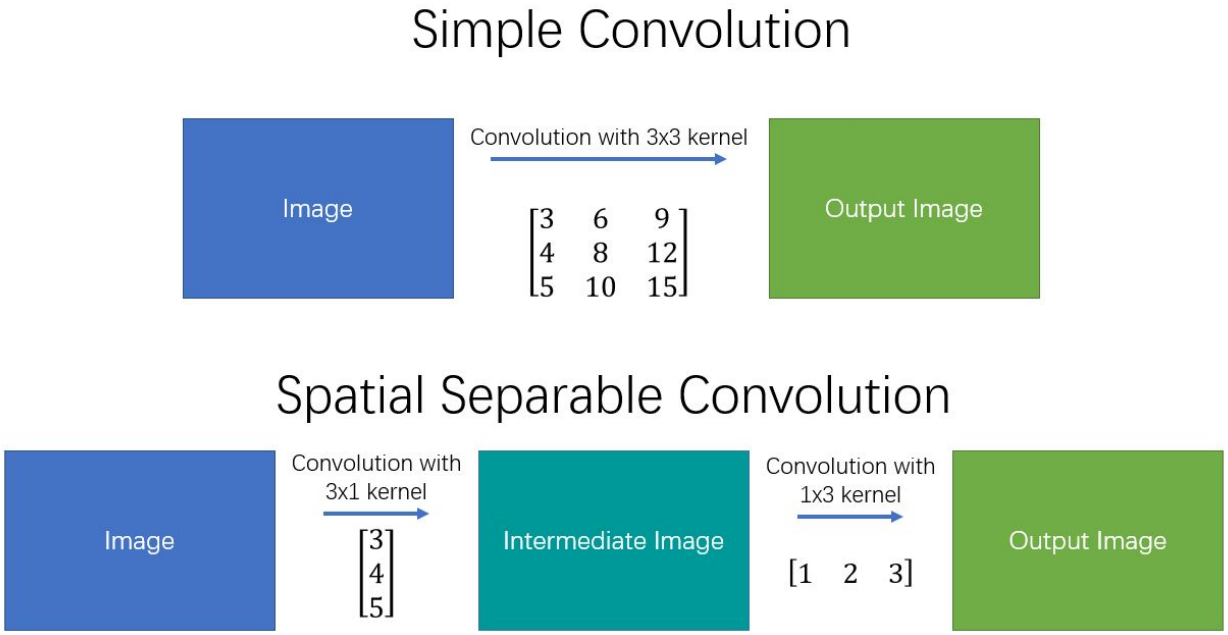
Separable convs
MobileNet

Spatial Separable Convolution

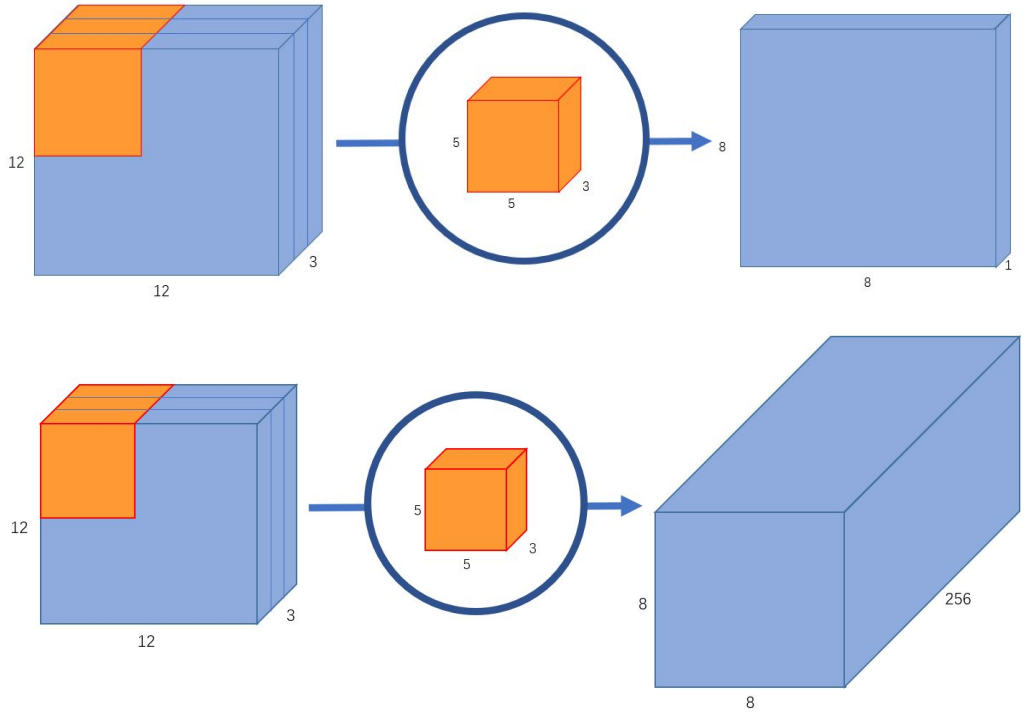
$$\begin{bmatrix} 3 & 6 & 9 \\ 4 & 8 & 12 \\ 5 & 10 & 15 \end{bmatrix} = \begin{bmatrix} 3 \\ 4 \\ 5 \end{bmatrix} \times \begin{bmatrix} 1 & 2 & 3 \end{bmatrix}$$

Images credentials

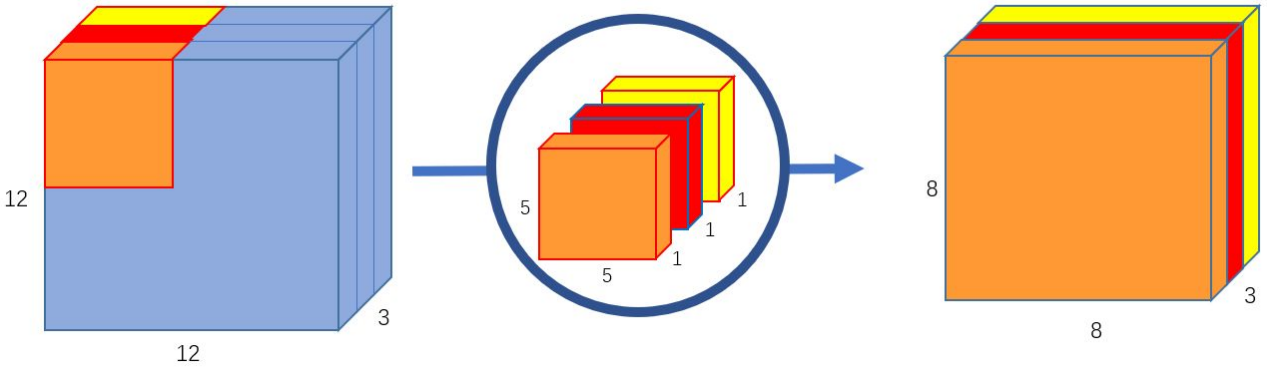
Spatial Separable Convolution



Normal Convolution

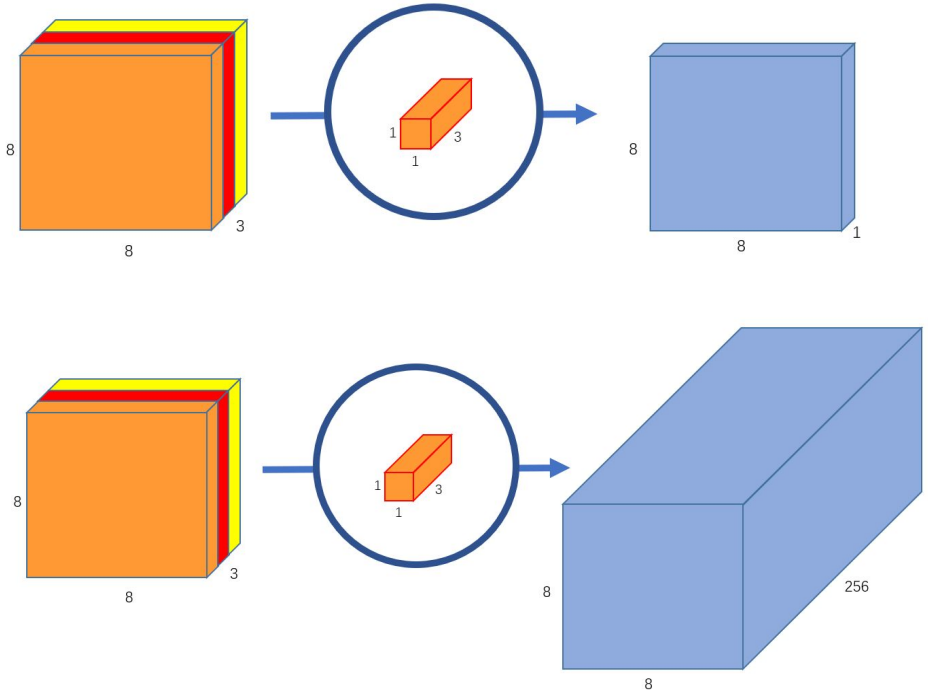


Depthwise Convolution



Don’t change channels number

Pointwise Convolution



MobileNet v1

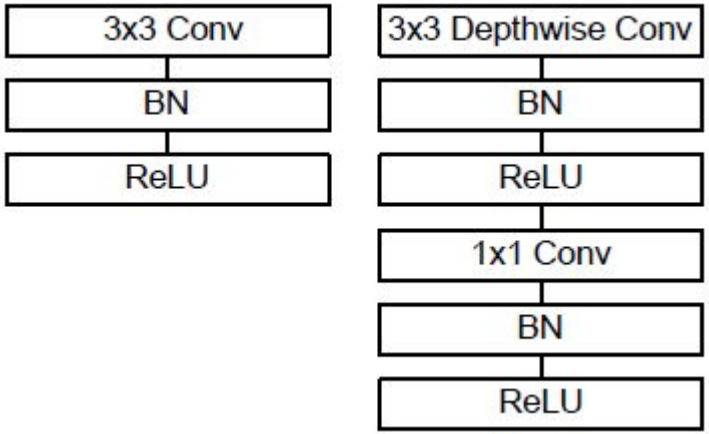
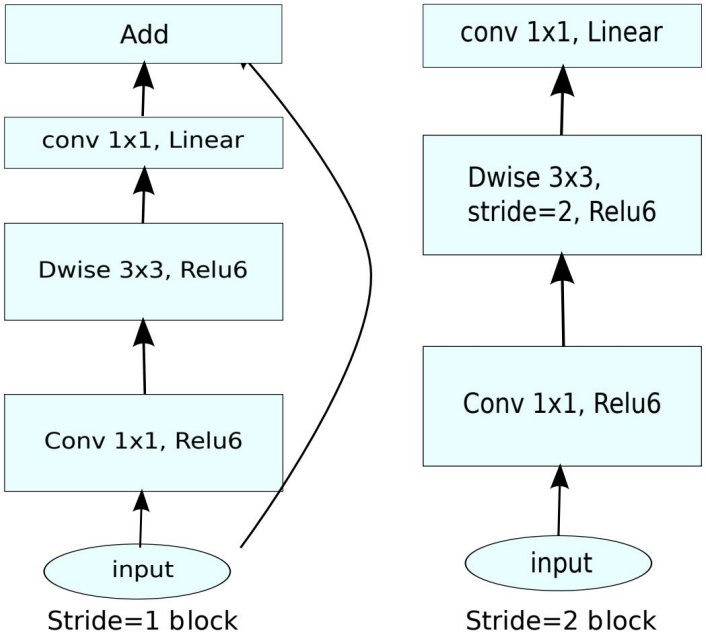
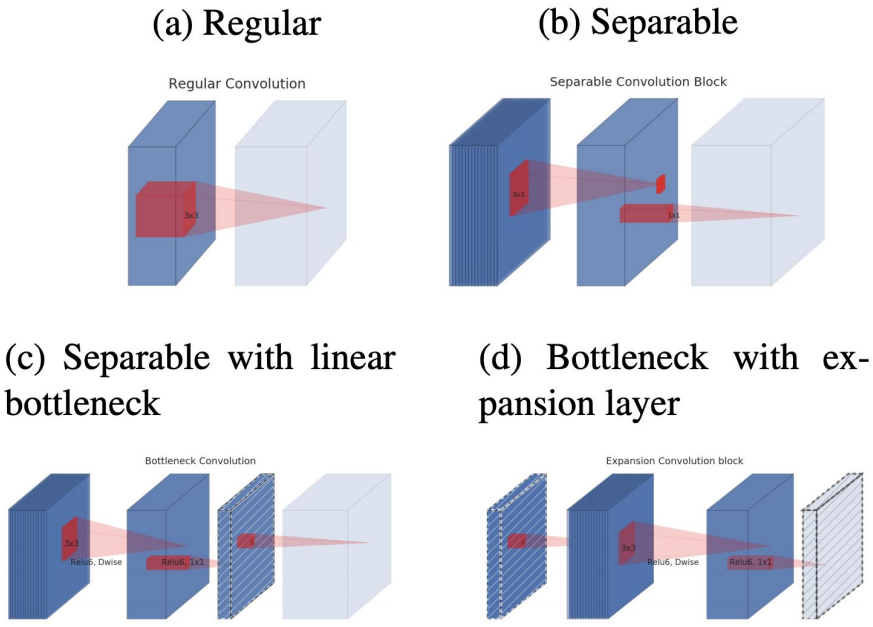


Table 1. MobileNet Body Architecture

Type / Stride	Filter Shape	Input Size
Conv / s2	$3 \times 3 \times 3 \times 32$	$224 \times 224 \times 3$
Conv dw / s1	$3 \times 3 \times 32$ dw	$112 \times 112 \times 32$
Conv / s1	$1 \times 1 \times 32 \times 64$	$112 \times 112 \times 32$
Conv dw / s2	$3 \times 3 \times 64$ dw	$112 \times 112 \times 64$
Conv / s1	$1 \times 1 \times 64 \times 128$	$56 \times 56 \times 64$
Conv dw / s1	$3 \times 3 \times 128$ dw	$56 \times 56 \times 128$
Conv / s1	$1 \times 1 \times 128 \times 128$	$56 \times 56 \times 128$
Conv dw / s2	$3 \times 3 \times 128$ dw	$56 \times 56 \times 128$
Conv / s1	$1 \times 1 \times 128 \times 256$	$28 \times 28 \times 128$
Conv dw / s1	$3 \times 3 \times 256$ dw	$28 \times 28 \times 256$
Conv / s1	$1 \times 1 \times 256 \times 256$	$28 \times 28 \times 256$
Conv dw / s2	$3 \times 3 \times 256$ dw	$28 \times 28 \times 256$
Conv / s1	$1 \times 1 \times 256 \times 512$	$14 \times 14 \times 256$
$5\times$ Conv dw / s1	$3 \times 3 \times 512$ dw	$14 \times 14 \times 512$
$5\times$ Conv / s1	$1 \times 1 \times 512 \times 512$	$14 \times 14 \times 512$
Conv dw / s2	$3 \times 3 \times 512$ dw	$14 \times 14 \times 512$
Conv / s1	$1 \times 1 \times 512 \times 1024$	$7 \times 7 \times 512$
Conv dw / s2	$3 \times 3 \times 1024$ dw	$7 \times 7 \times 1024$
Conv / s1	$1 \times 1 \times 1024 \times 1024$	$7 \times 7 \times 1024$
Avg Pool / s1	Pool 7×7	$7 \times 7 \times 1024$
FC / s1	1024×1000	$1 \times 1 \times 1024$
Softmax / s1	Classifier	$1 \times 1 \times 1000$

MobileNet v2





Results

Architecture	Params	**Top-1 accuracy**	**Top-5 accuracy**
Xception	22.91M	0.790	0.945
VGG16	138.35M	0.715	0.901
MobileNetV1 (alpha=1, rho=1)	4.20M	0.709	0.899
MobileNetV1 (alpha=0.75, rho=0.85)	2.59M	0.672	0.873
MobileNetV1 (alpha=0.25, rho=0.57)	0.47M	0.415	0.663
MobileNetV2 (alpha=1.4, rho=1)	6.06M	**0.750**	**0.925**
MobileNetV2 (alpha=1, rho=1)	3.47M	0.718	0.910
MobileNetV2 (alpha=0.35, rho=0.43)	1.66M	0.455	0.704

MobileNet v3

Layer removal

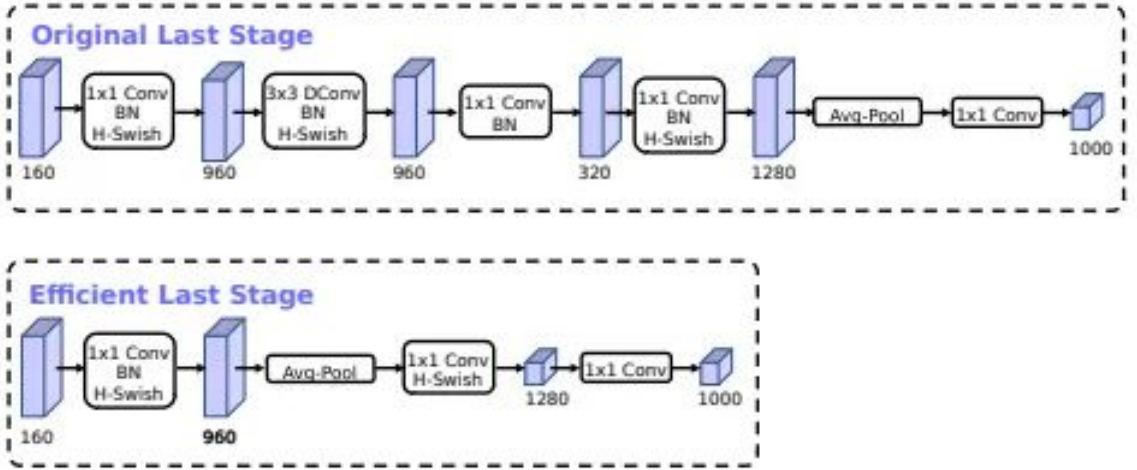


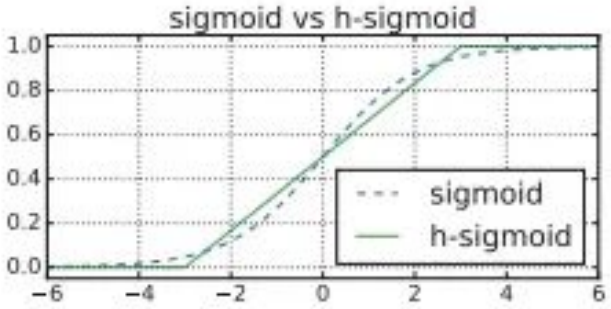
Figure 5. Comparison of original last stage and efficient last stage. This more efficient last stage is able to drop three expensive layers at the end of the network at no loss of accuracy.

MobileNet v3

Nonlinearity

$$\text{swish}\, x = x \cdot \sigma(x)$$

$$\text{h-swish}[x] = x \frac{\text{ReLU6}(x+3)}{6}$$



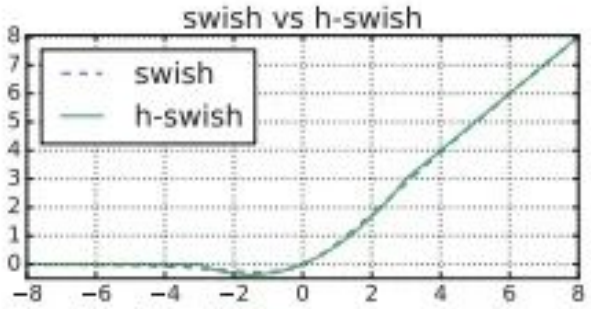


Figure 6. Sigmoid and swish nonlinearities and ther "hard" counterparts.

Results

Network	Top-1	MAdds	Params	P-1	P-2	P-3
V3-Large 1.0	**75.2**	**219**	5.4M	**51**	**61**	**44**
V3-Large 0.75	73.3	155	4.0M	39	46	40
MnasNet-A1	75.2	315	3.9M	71	86	61
Proxyless[5]	74.6	320	4.0M	72	84	60
V2 1.0	72.0	300	3.4M	64	76	56
V3-Small 1.0	**67.4**	**56**	2.5M	**15.8**	**19.4**	**14.4**
V3-Small 0.75	65.4	44	2.0M	12.8	15.6	11.7
Mnas-small [43]	64.9	65.1	1.9M	20.3	24.2	17.2
V2 0.35	60.8	59.2	1.6M	16.6	19.6	13.9

Table 3. Floating point performance on the Pixel family of phones (P-n denotes a Pixel-n phone). All latencies are in ms and are measured using a single large core with a batch size of one. Top-1 accuracy is on ImageNet.

Revise

- Recap
- Focal Loss
- Non-Maximum Suppression
- YOLO
- Separable convolutions
- MobileNet

Next time

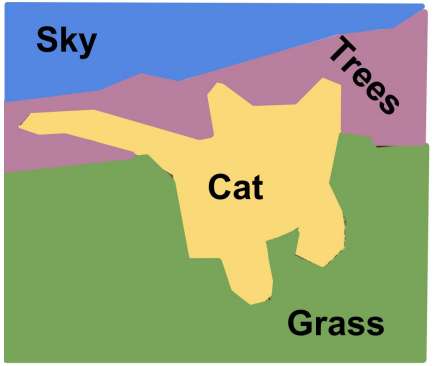
- Segmentation

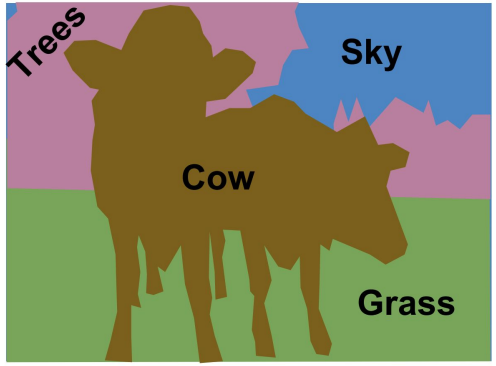
See [Mask R-CNN ICCV17 talk] to prepare yourself

Outline

- Segmentation task
- Simple solutions
- Upsampling methods
  - Unpooling
  - Transposed convolution
- FCN
- DeconvNet
- SegNet
- U-Net
- Mask R-CNN

Semantic Segmentation





Input image (C x H x W)

Output

mask of classes (1 x H x W)

Assumption

just class label for each pixel

Instance Segmentation

Input image (C x H x W)

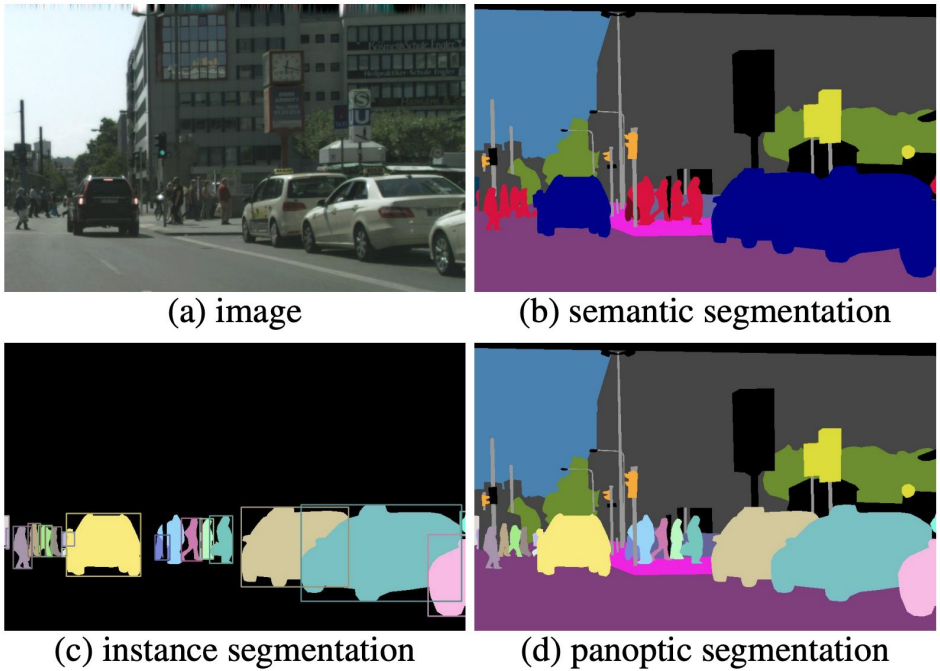
Output

mask of class instances (detection + 1 x H x W)

Assumption

separated masks for each distinct object

Panoptic segmentation



(a) image (b) semantic segmentation

(c) instance segmentation (d) panoptic segmentation

Input image (C x H x W)

Output

mask of class instances (detection + 1 x H x W)

Assumption

separated masks for each distinct object

https://arxiv.org/pdf/1801.00868.pdf

https://viso.ai/deep-learning/panoptic-segmentation/

Panoptic links

- http://presentations.cocodataset.org/ECCV18/COCO18-Panoptic-PKU_360.pdf
- https://viso.ai/deep-learning/panoptic-segmentation-a-basic-to-advanced-guide-2024/

Metrics

- IoU
- Pixel accuracy
- Average Precision (AP)

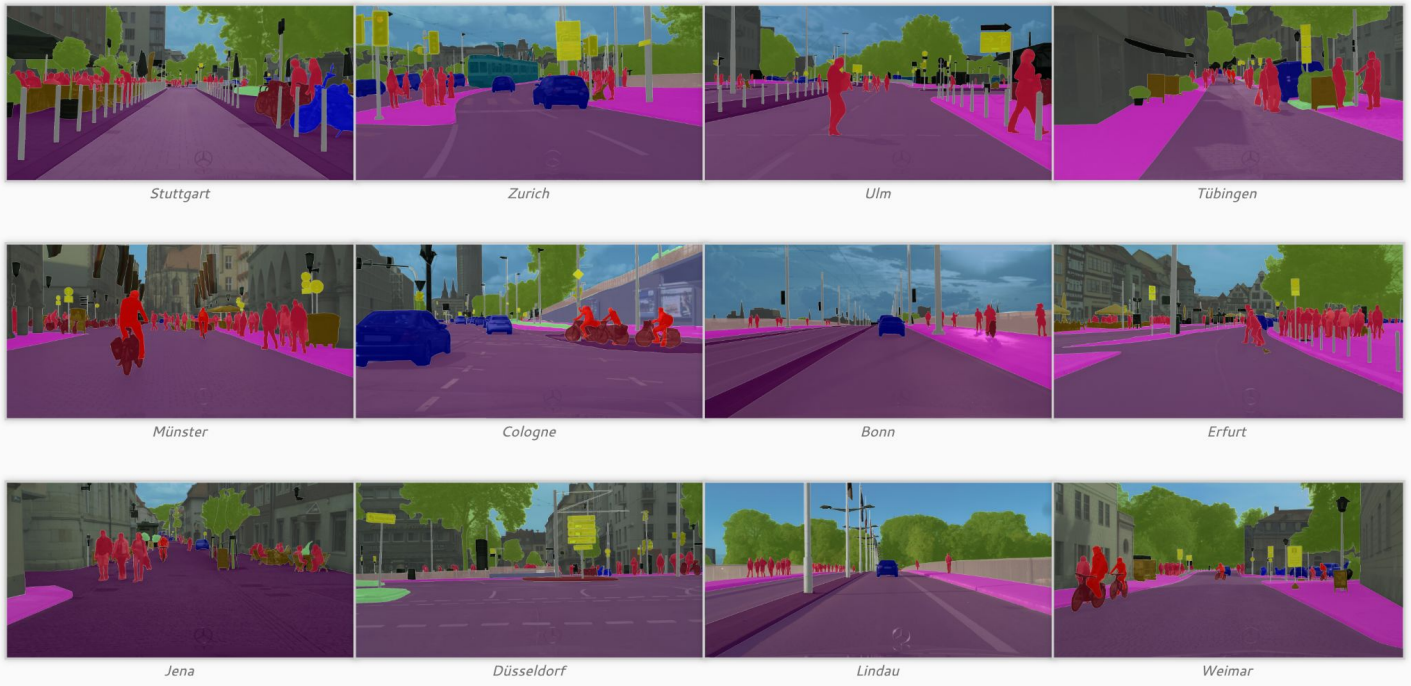
Datasets

COCO Panoptic Segmentation

https://cocodataset.org/#panoptic-2018

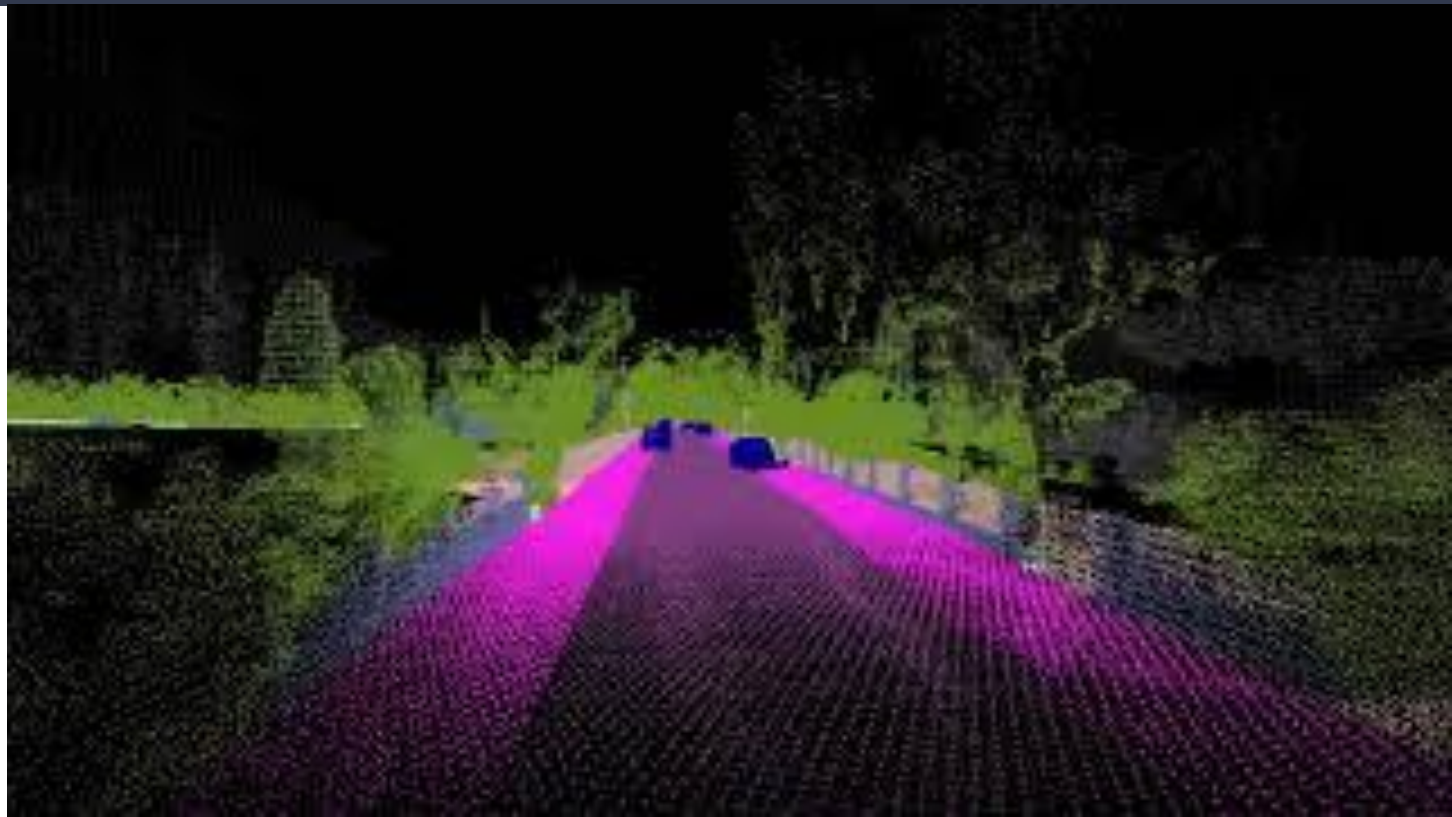
Cityscapes

https://www.cityscapes-dataset.com/



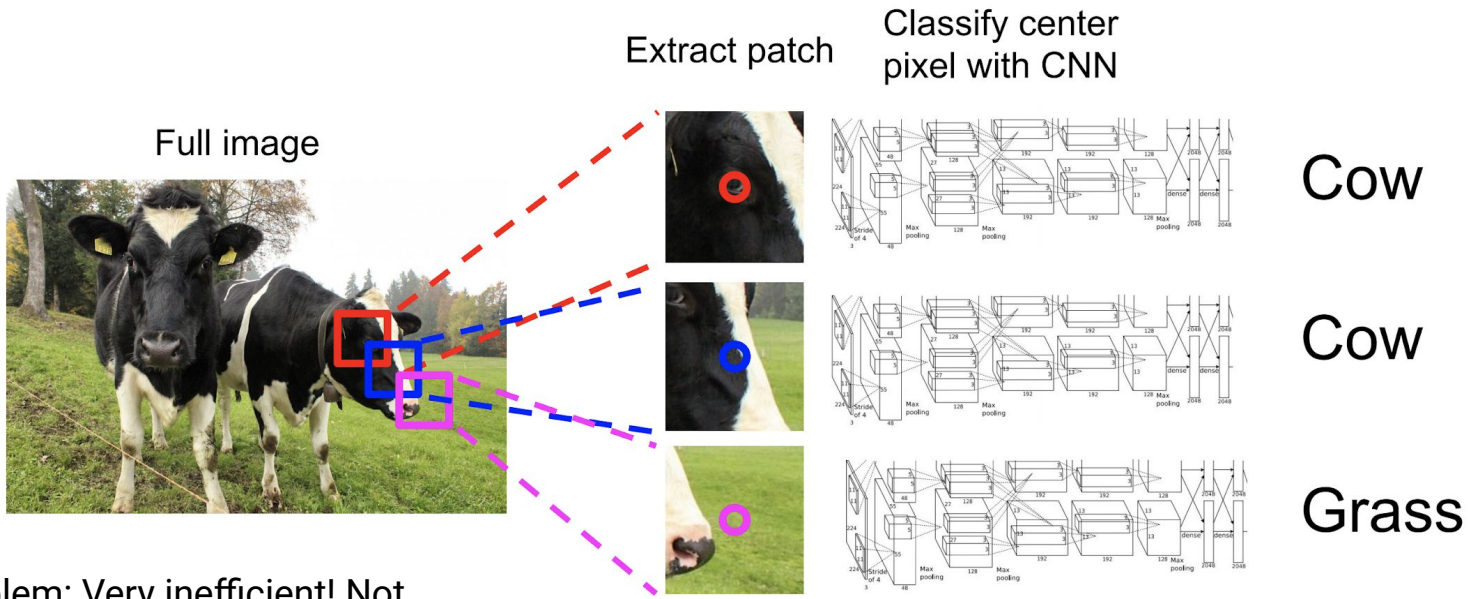
KITTI

https://www.cvlibs.net/datasets/kitti-360/



Models

Naive solution

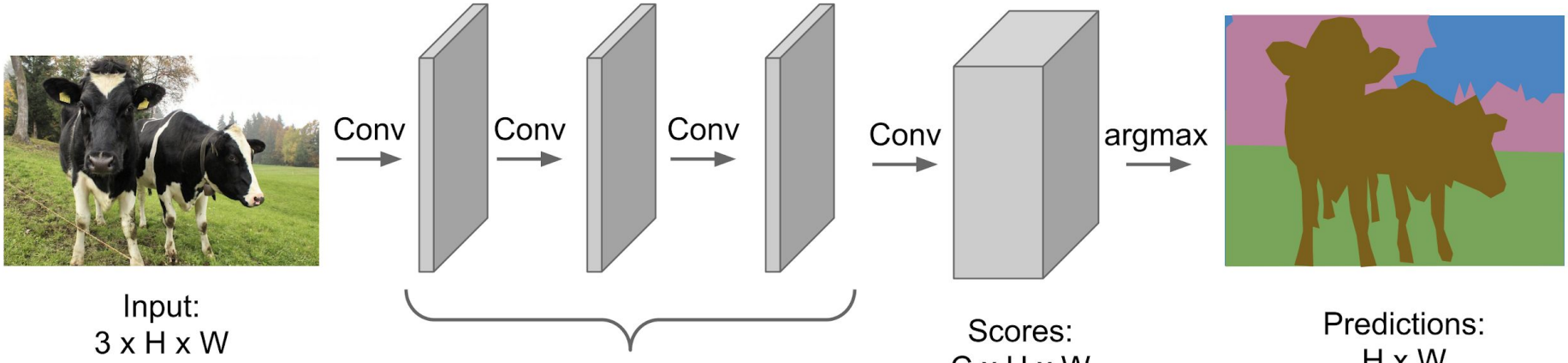


Problem: Very inefficient! Not reusing shared features between overlapping patches (same as R-CNN)

Farabet et al, “Learning Hierarchical Features for Scene Labeling,” TPAMI 2013
Pinheiro and Collobert, “Recurrent Convolutional Neural Networks for Scene Labeling”, ICML 2014

Fully Convolutional

Design a network as a bunch of convolutional layers to make predictions for pixels all at once!

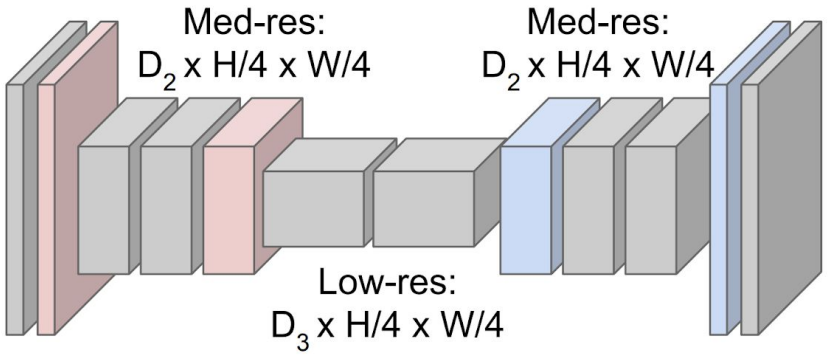


Problem: convolutions at original image resolution will be very expensive (remember VGG architecture)

Downsampling and Upsampling

Design network as a bunch of convolutional layers, with **downsampling** and **upsampling** inside the network!

Input:
3 x H x W

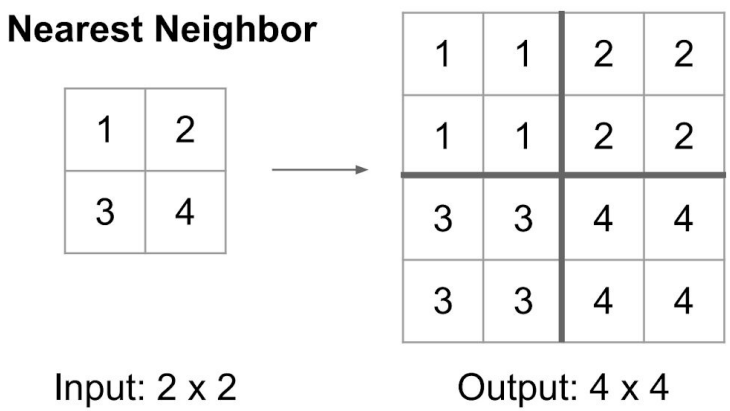


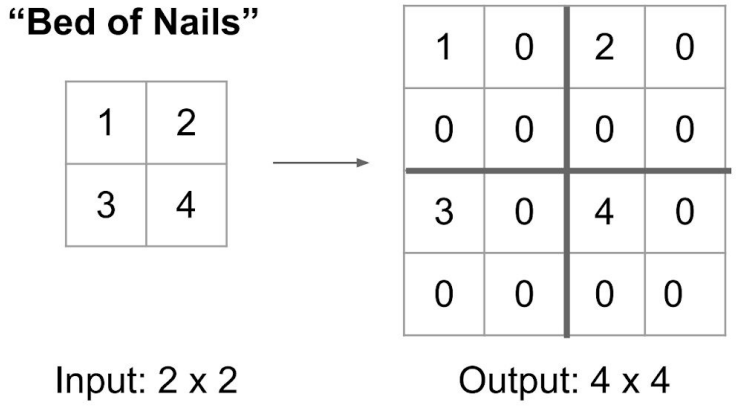
High-res:
D_1 x H/2 x W/2

High-res:
D_1 x H/2 x W/2

Predictions:
H x W

Simple unpooling





Max Unpooling

Max Pooling
Remember which element was max!

1	2	6	3
3	5	2	1
1	2	2	1
7	3	4	8

Input: 4 x 4

5	6
7	8

Output: 2 x 2

...

Rest of the network

Max Unpooling
Use positions from pooling layer

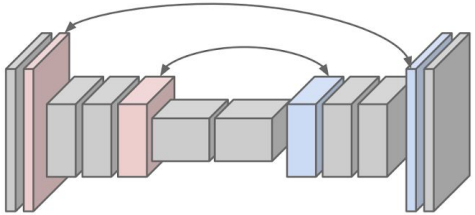
1	2
3	4

Input: 2 x 2

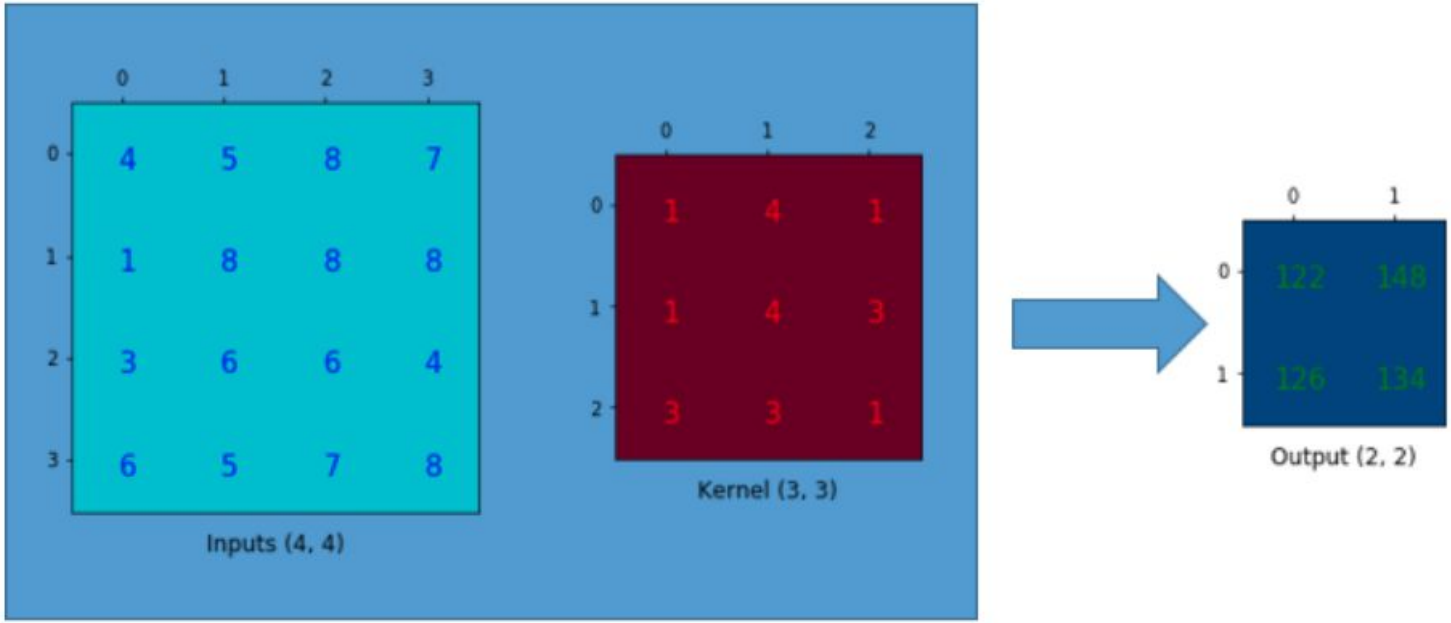
0	0	2	0
0	1	0	0
0	0	0	0
3	0	0	4

Output: 4 x 4

Corresponding pairs of downsampling and upsampling layers

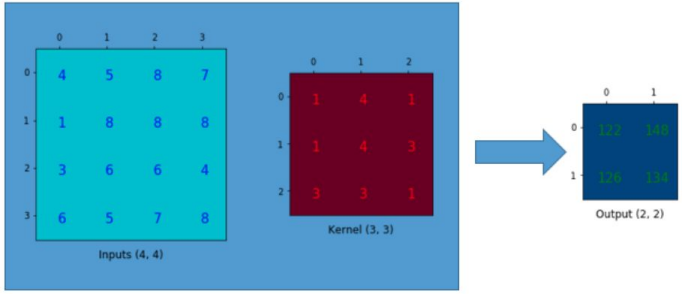


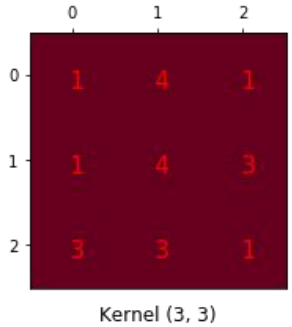
Regular convolution

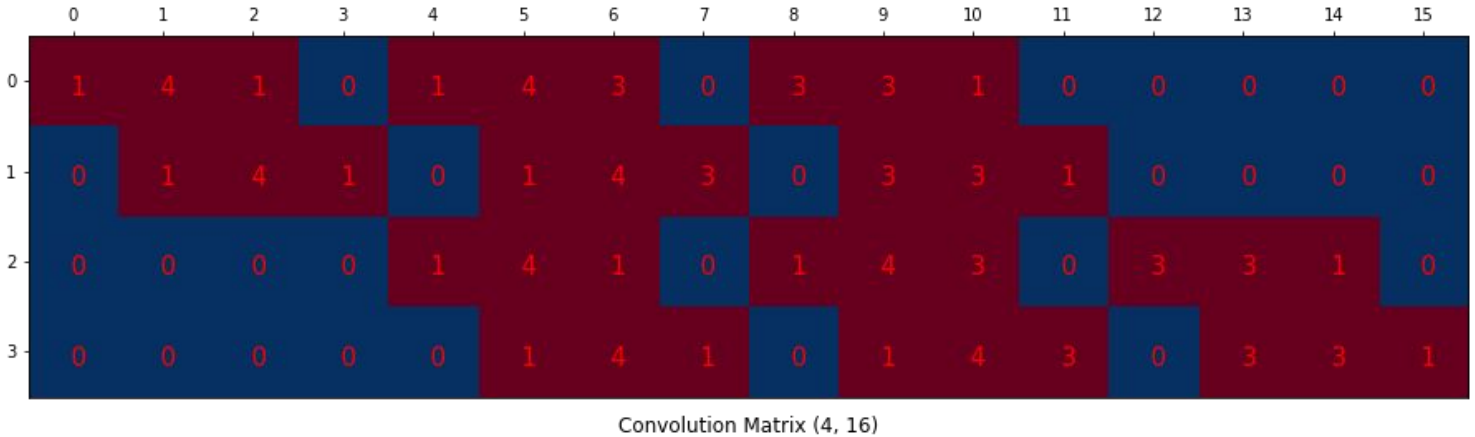


Images credentials

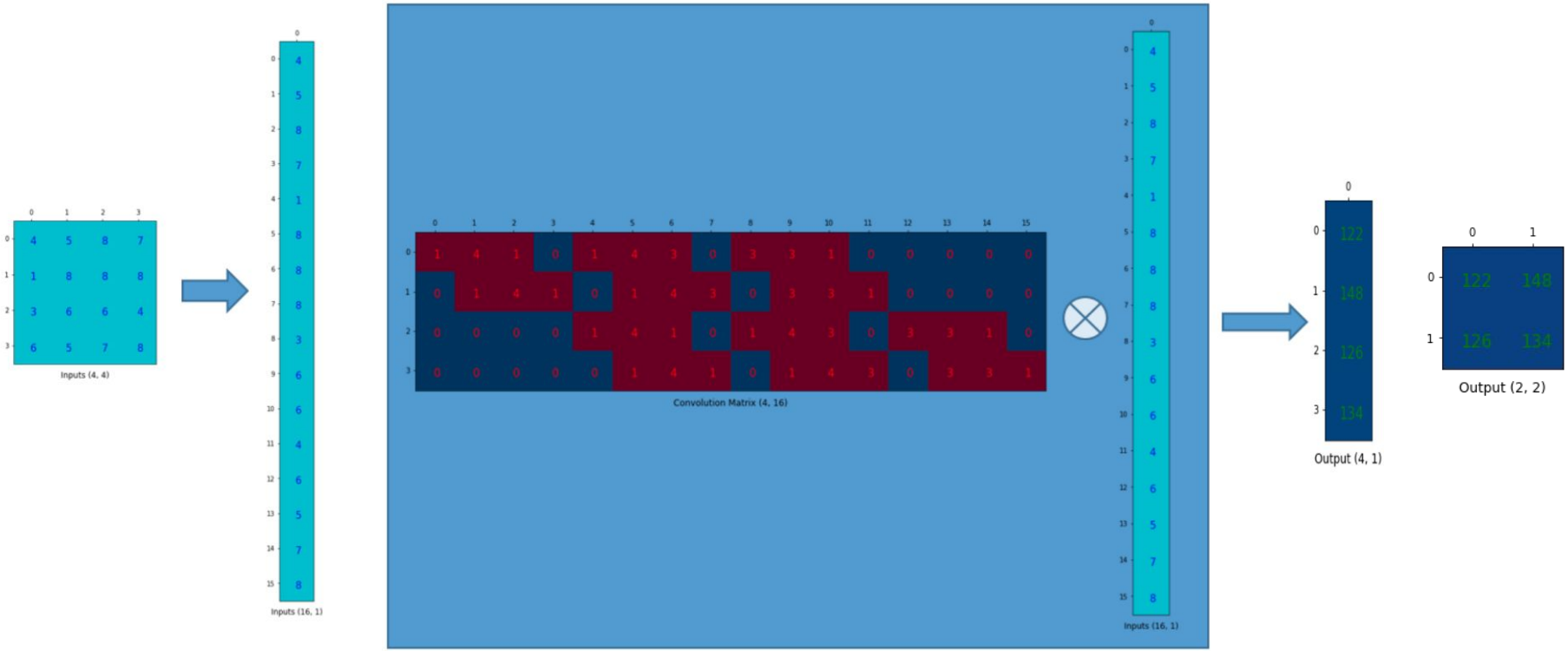
Regular convolution: matrix form



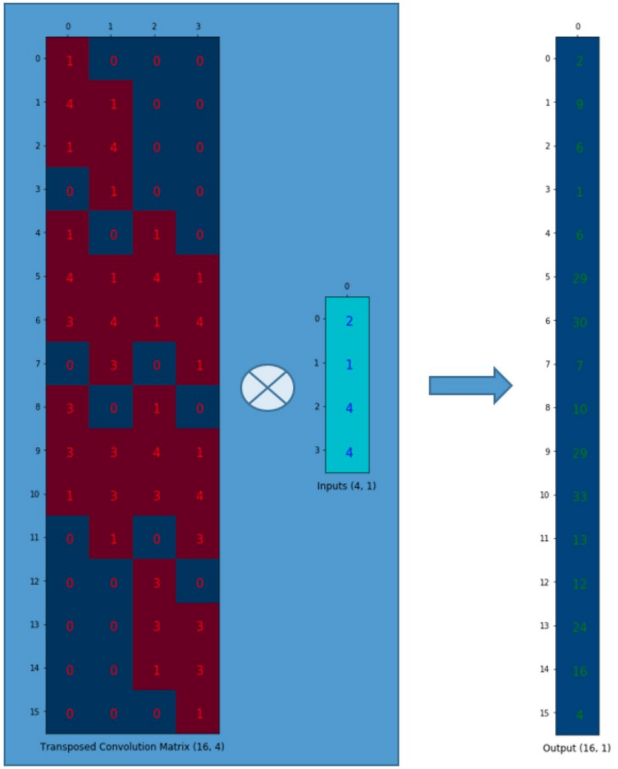


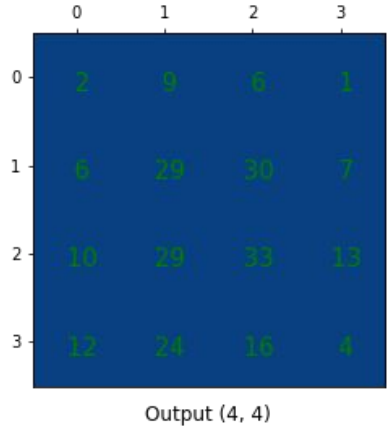


Regular convolution: matrix form



Transposed convolution matrix





Convolution as Matrix Multiplication (1D)

We can express convolution in terms of a matrix multiplication

$$\vec{x} * \vec{a} = X\vec{a}$$

$$\begin{bmatrix} x & y & x & 0 & 0 & 0 \\ 0 & x & y & x & 0 & 0 \\ 0 & 0 & x & y & x & 0 \\ 0 & 0 & 0 & x & y & x \end{bmatrix} \begin{bmatrix} 0 \\ a \\ b \\ c \\ d \\ 0 \end{bmatrix} = \begin{bmatrix} ay + bz \\ ax + by + cz \\ bx + cy + dz \\ cx + dy \end{bmatrix}$$

Example: 1D conv, kernel size=3, stride=1, padding=1

Convolution transpose multiplies by the transpose of the same matrix:

$$\vec{x} *^T \vec{a} = X^T\vec{a}$$

$$\begin{bmatrix} x & 0 & 0 & 0 \\ y & x & 0 & 0 \\ z & y & x & 0 \\ 0 & z & y & x \\ 0 & 0 & z & y \\ 0 & 0 & 0 & z \end{bmatrix} \begin{bmatrix} a \\ b \\ c \\ d \end{bmatrix} = \begin{bmatrix} ax \\ ay + bx \\ az + by + cx \\ bz + cy + dx \\ cz + dy \\ dz \end{bmatrix}$$

When stride=1, convolution transpose is just a regular convolution (with different padding rules)

Terms

Some researchers denote Transposed convolution with term “Deconvolution”. However:

deconvolution is an algorithm-based process used to reverse the effects of convolution on recorded data [Wikipedia]

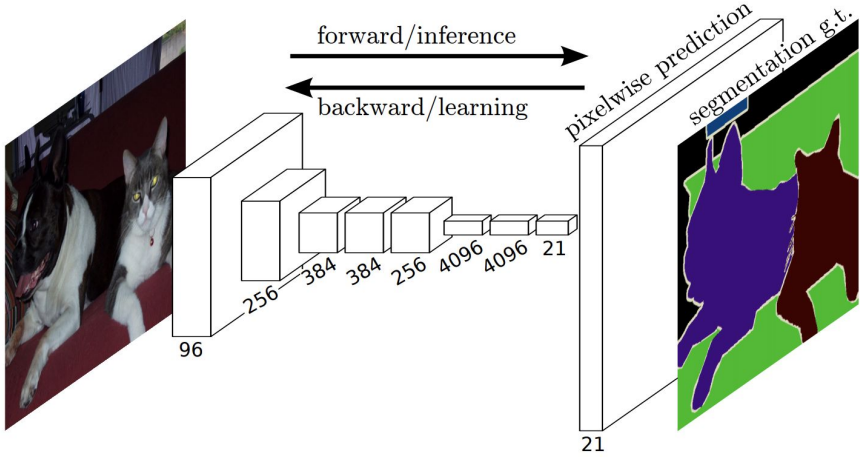
What we do is not a deconvolution.

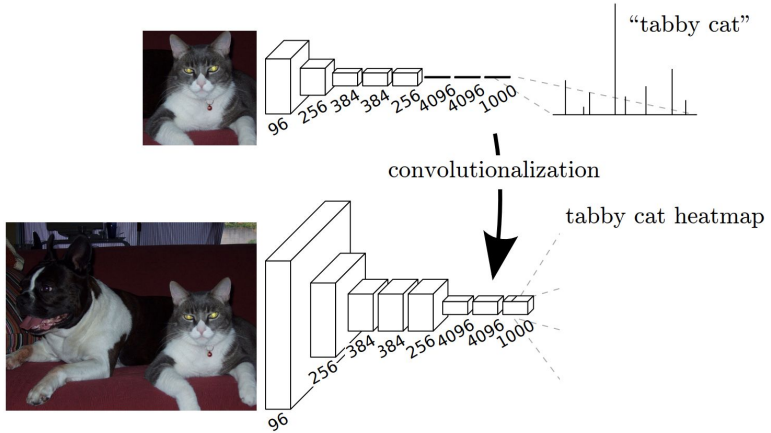
Please be precise in terms

Right terms:

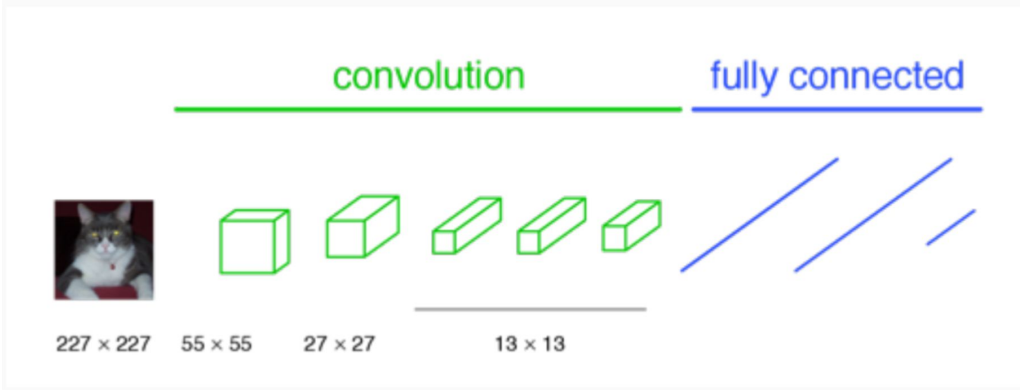
- Transposed convolution
- Upconvolution

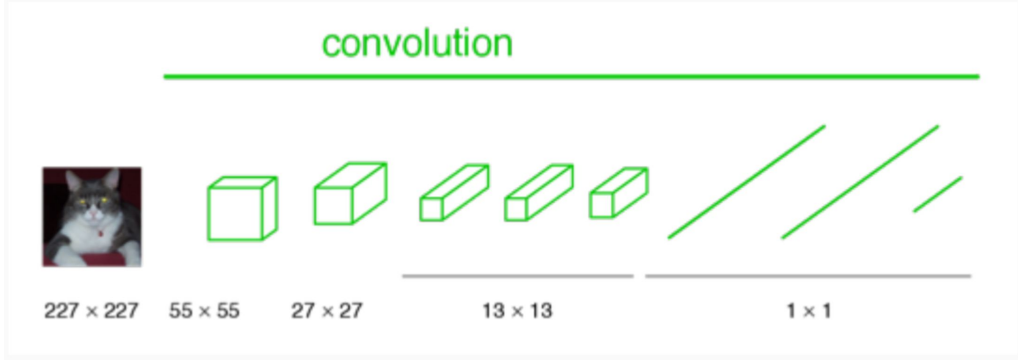
FCN – Fully Convolutional Network



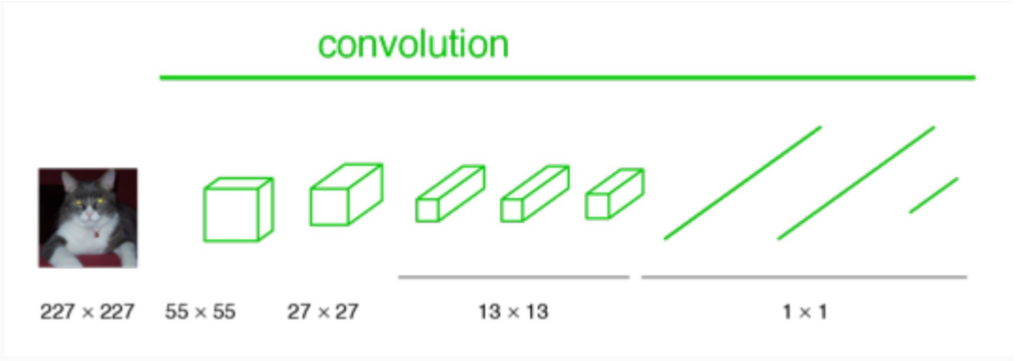


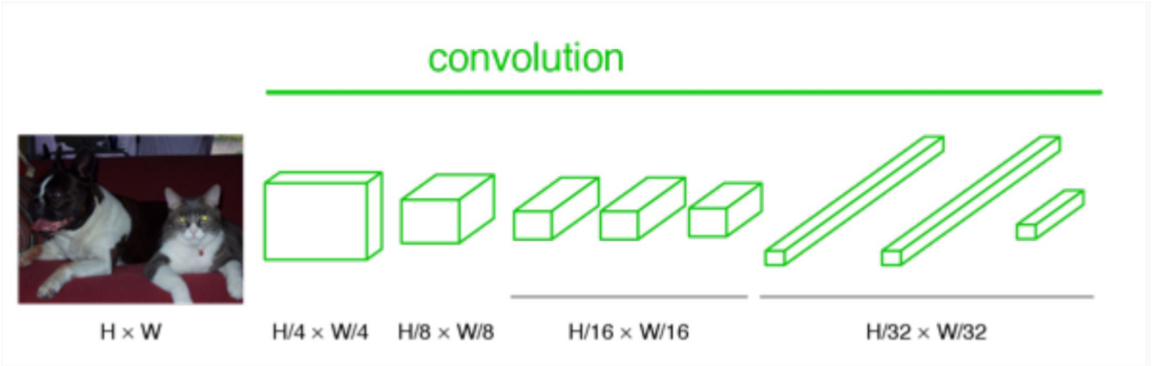
Regular CNN to FCN



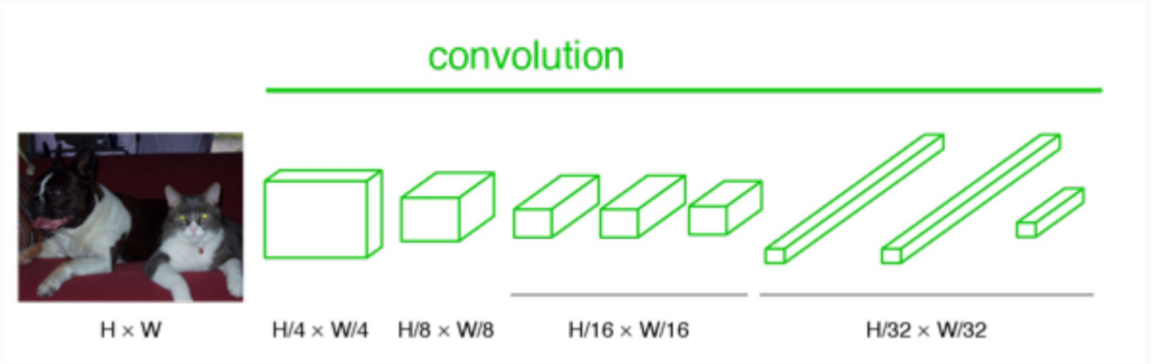


Regular CNN to FCN



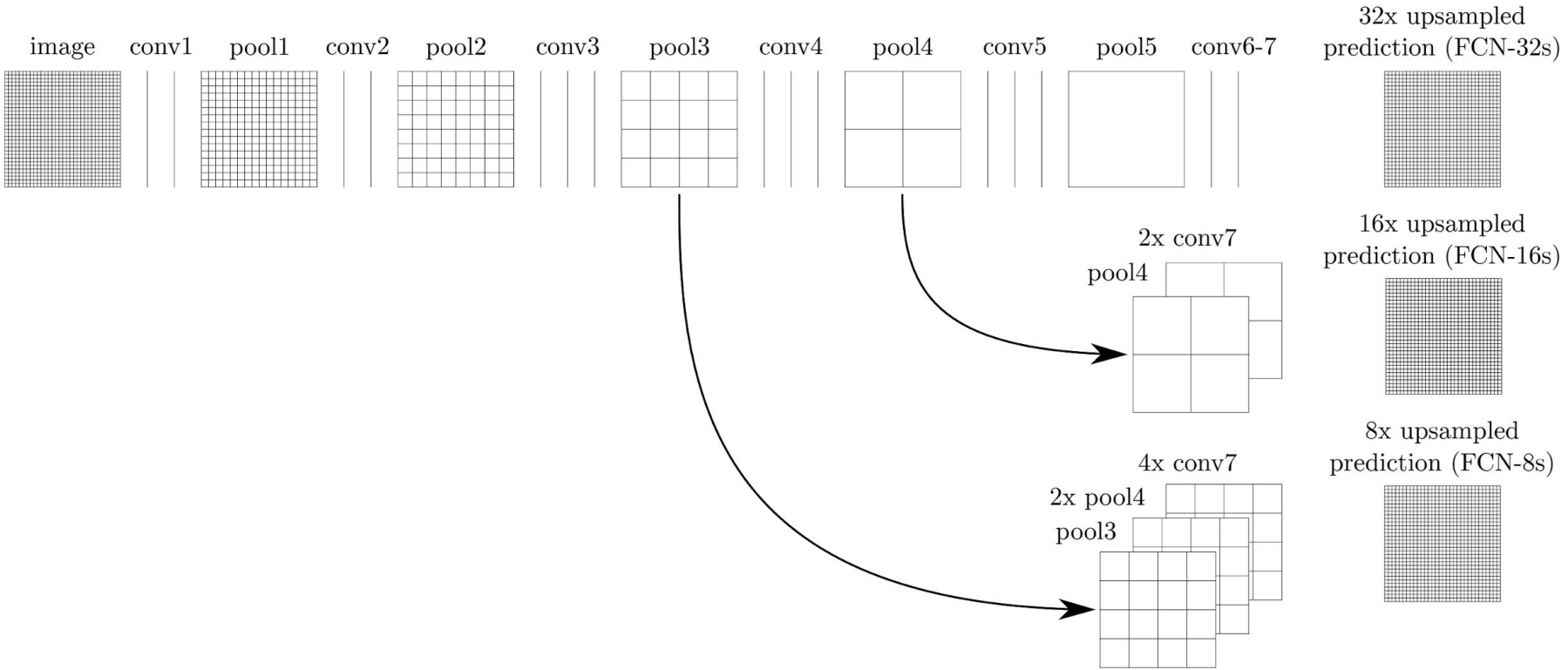


Regular CNN to FCN

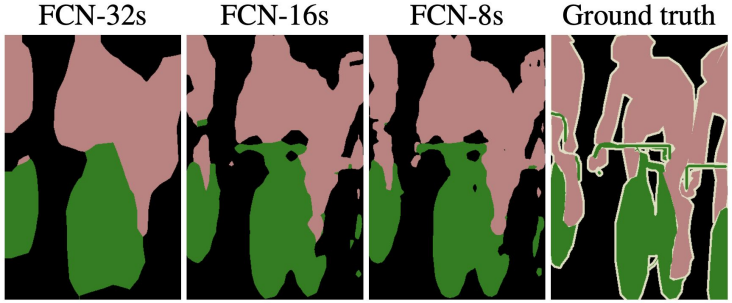


Problem: we lose resolution

FCN: Introduce skip connections

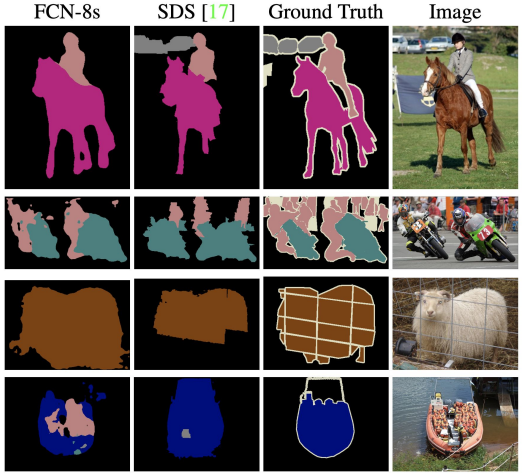


FCN: Results

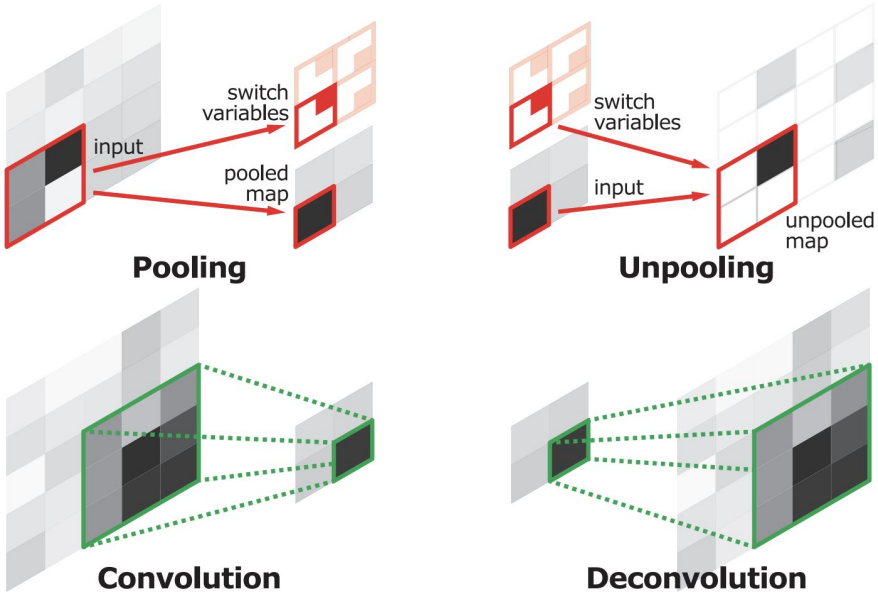


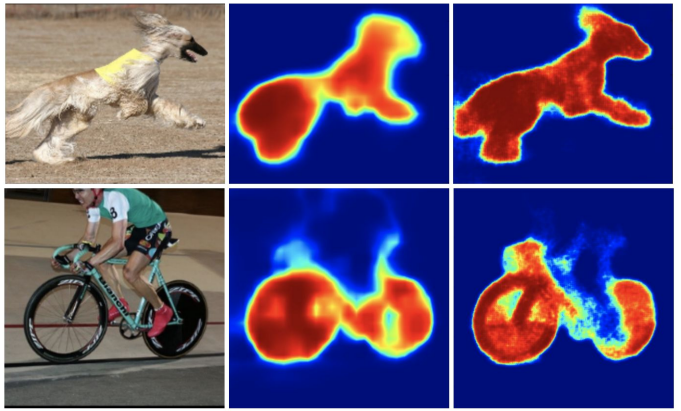
	pixel acc.	mean acc.	mean IU	f.w. IU
FCN-32s-fixed	83.0	59.7	45.4	72.0
FCN-32s	89.1	73.3	59.4	81.4
FCN-16s	90.0	75.7	62.4	83.0
FCN-8s	**90.3**	**75.9**	**62.7**	**83.2**

	FCN-AlexNet	FCN-VGG16	FCN-GoogLeNet[4]
mean IU	39.8	**56.0**	42.5
forward time	50 ms	210 ms	59 ms
conv. layers	8	16	22
parameters	57M	134M	6M
rf size	355	404	907
max stride	32	32	32



DeconvNet: use Max Unpooling





(a) Input image (b) FCN-8s (c) Ours

Figure 5. Comparison of class conditional probability maps from FCN and our network (top: dog, bottom: bicycle).

DeconvNet: use Max Unpooling

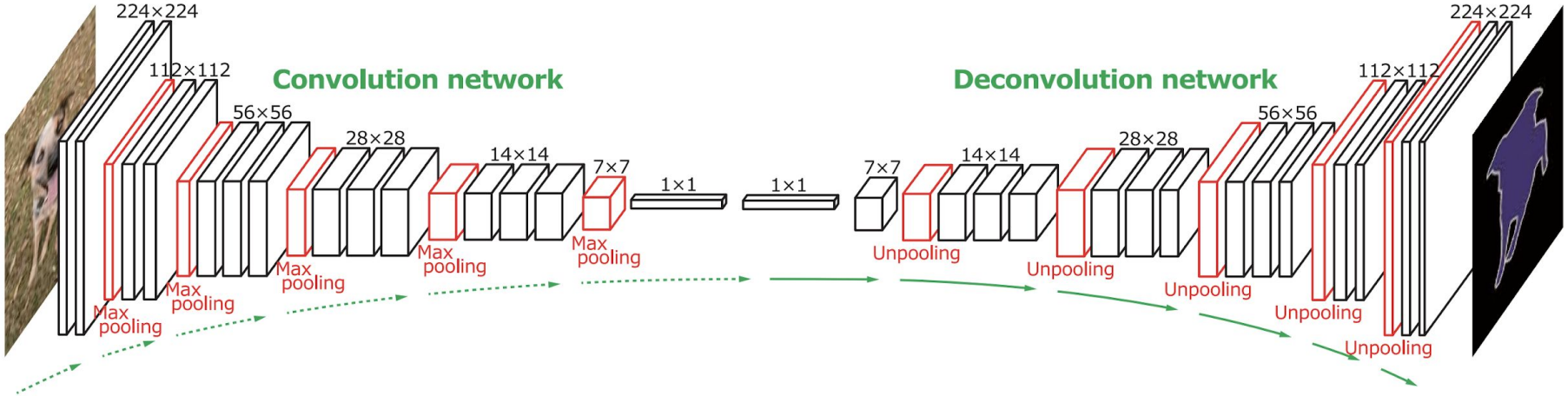
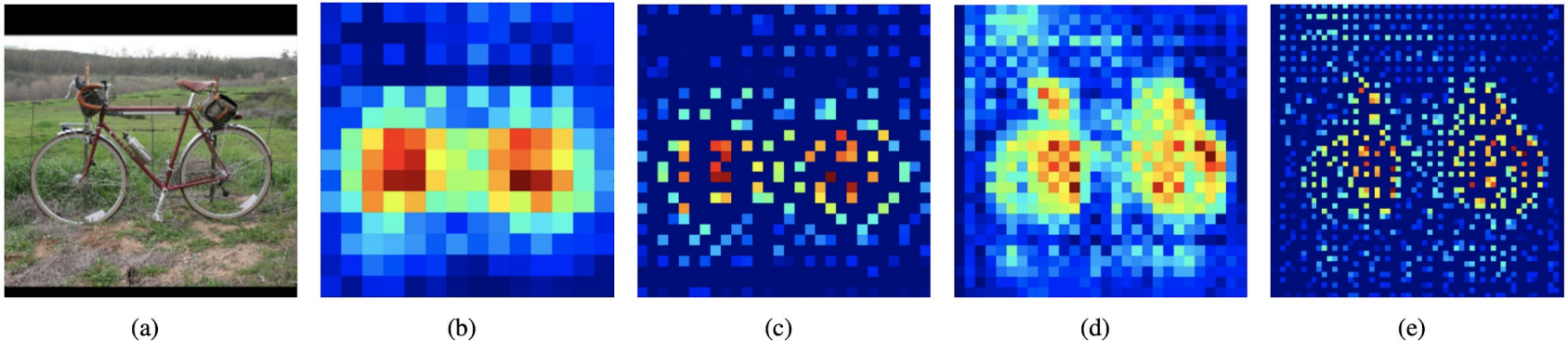


Figure 2. Overall architecture of the proposed network. On top of the convolution network based on VGG 16-layer net, we put a multi-layer deconvolution network to generate the accurate segmentation map of an input proposal. Given a feature representation obtained from the convolution network, dense pixel-wise class prediction map is constructed through multiple series of unpooling, deconvolution and rectification operations.

DeconvNet: use Max Unpooling



(a) (b) (c) (d) (e)

(f) (g) (h) (i) (j)

SegNet: Further development

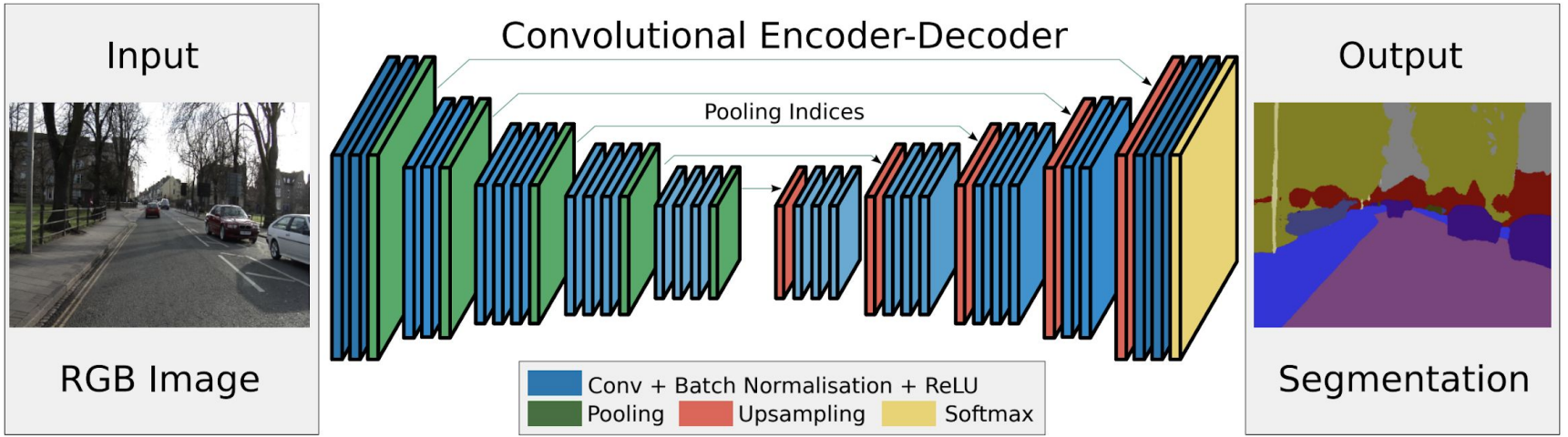
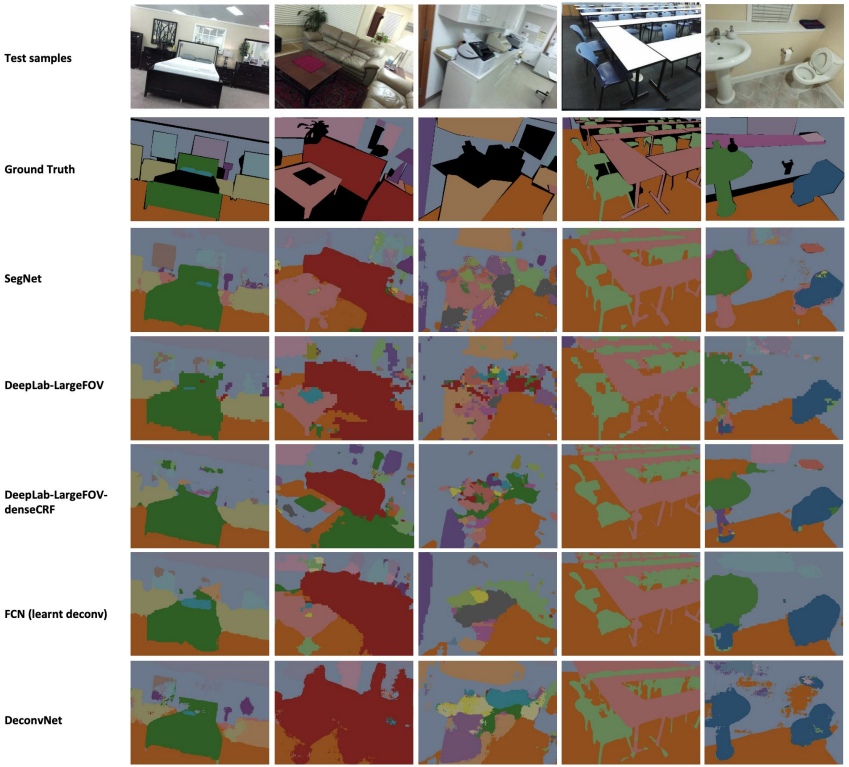


Fig. 2. An illustration of the SegNet architecture. There are no fully connected layers and hence it is only convolutional. A decoder upsamples its input using the transferred pool indices from its encoder to produce a sparse feature map(s). It then performs convolution with a trainable filter bank to densify the feature map. The final decoder output feature maps are fed to a soft-max classifier for pixel-wise classification.

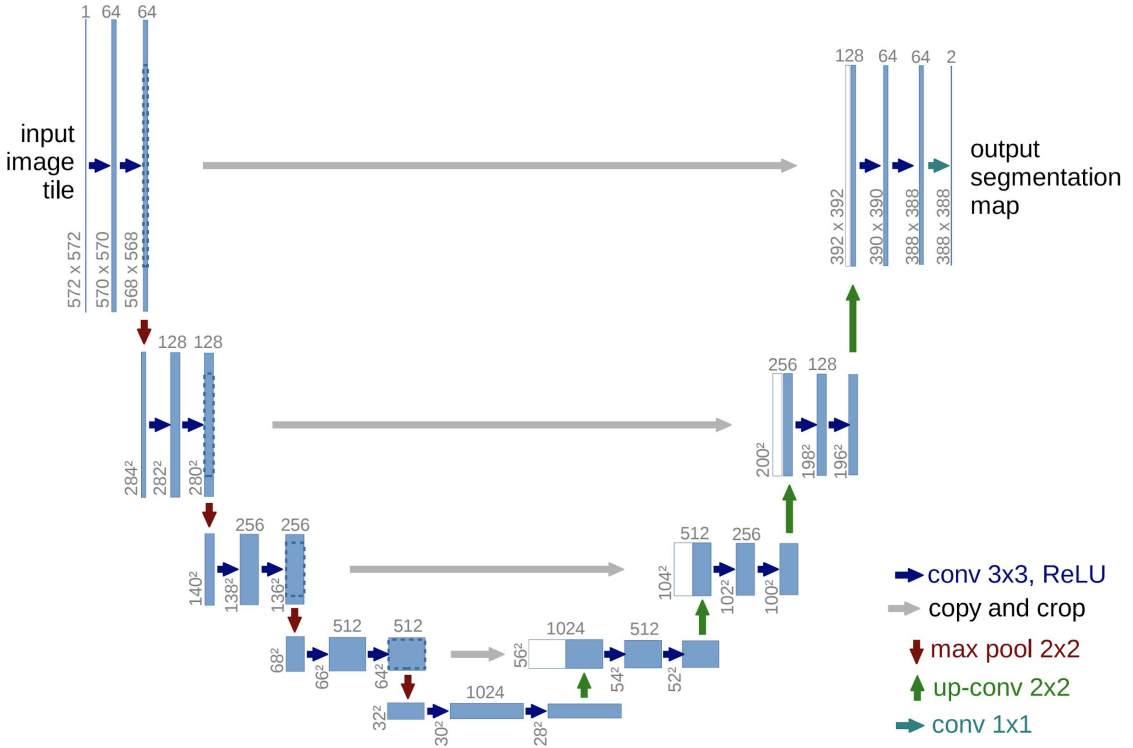
SegNet: Further development



U-Net

2015

https://arxiv.org/abs/1505.04597



U-Net

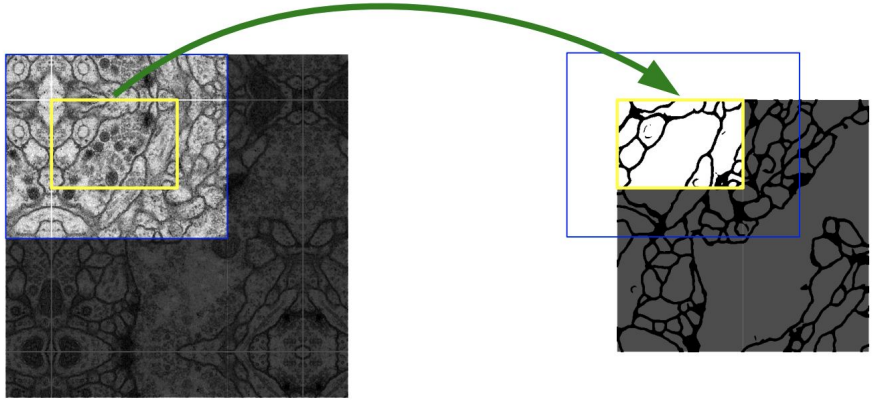
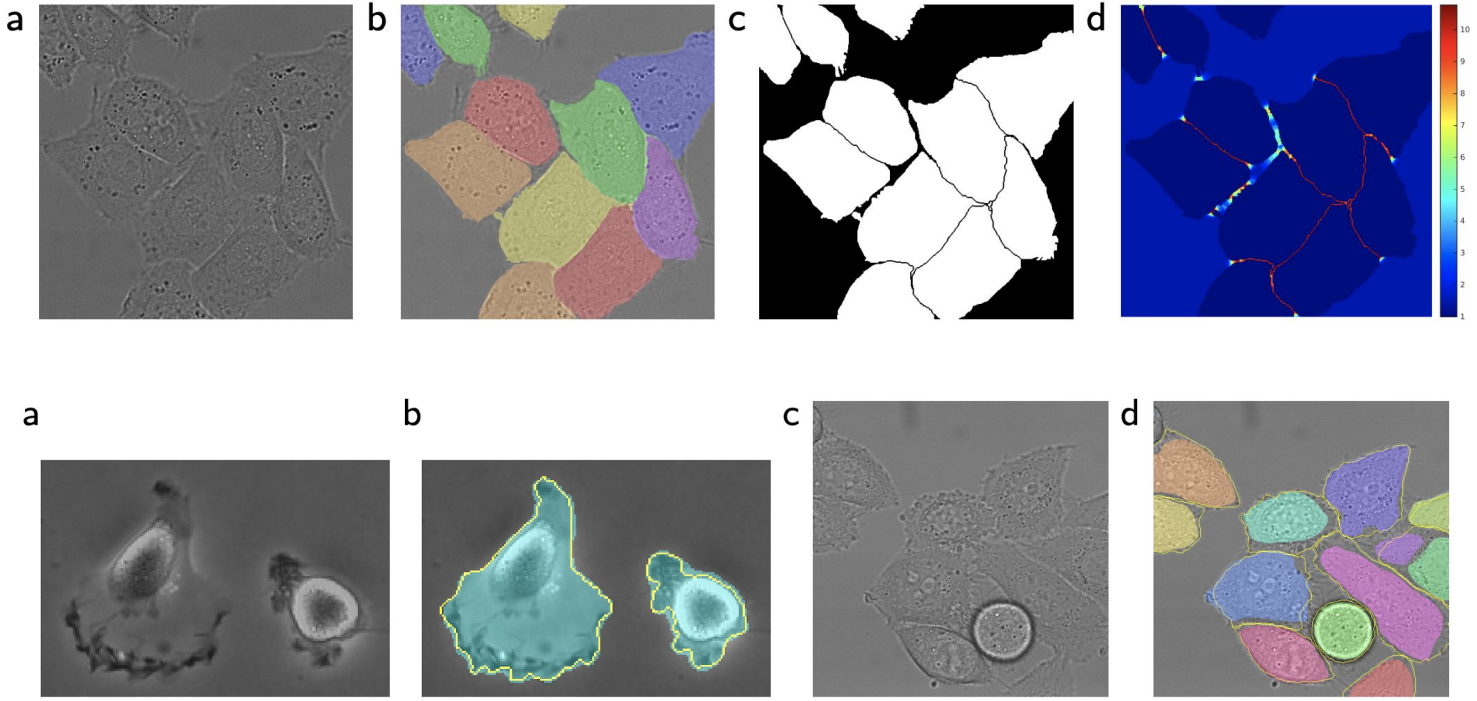


Fig. 2. Overlap-tile strategy for seamless segmentation of arbitrary large images (here segmentation of neuronal structures in EM stacks). Prediction of the segmentation in the yellow area, requires image data within the blue area as input. Missing input data is extrapolated by mirroring

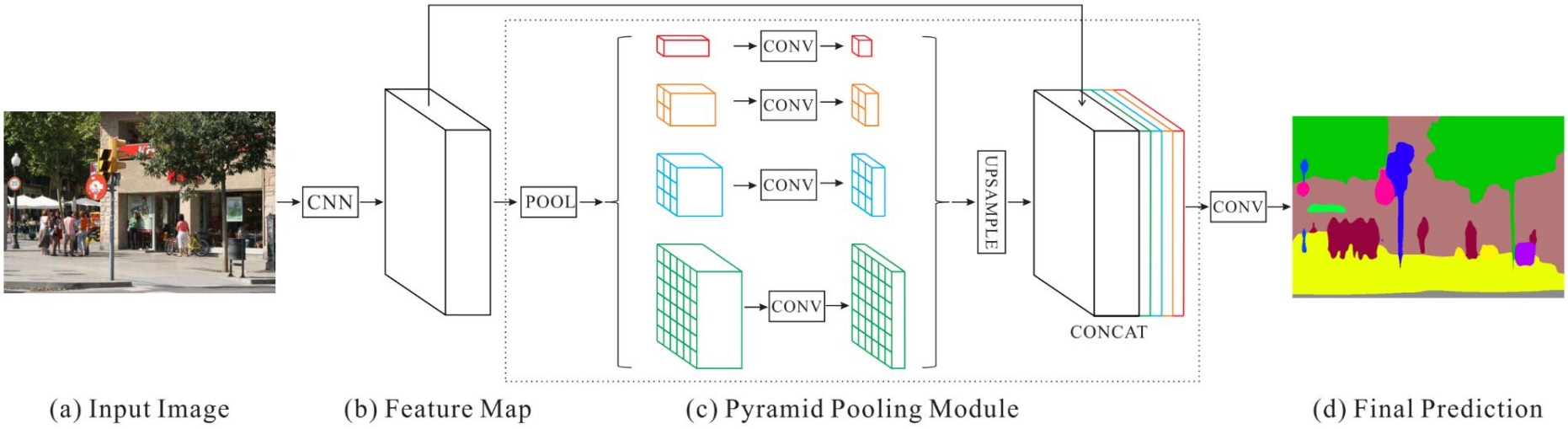
U-Net



PSPNet

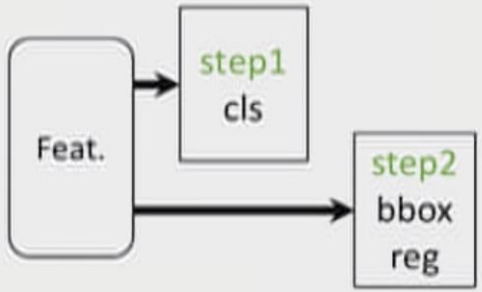
2017

https://arxiv.org/pdf/1612.01105.pdf

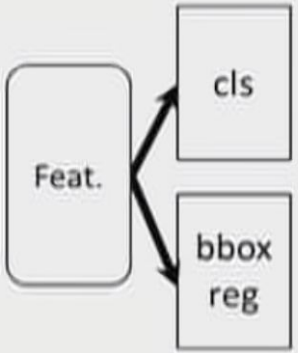


Mask R-CNN: Instance segmentation

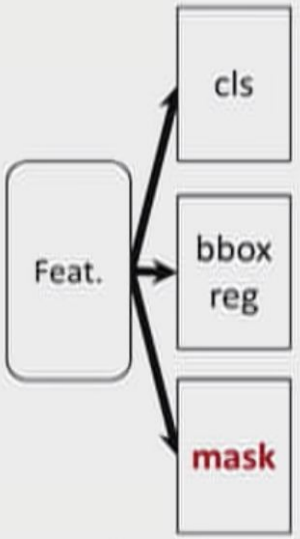
- Easy, fast to implement and use



(slow) R-CNN

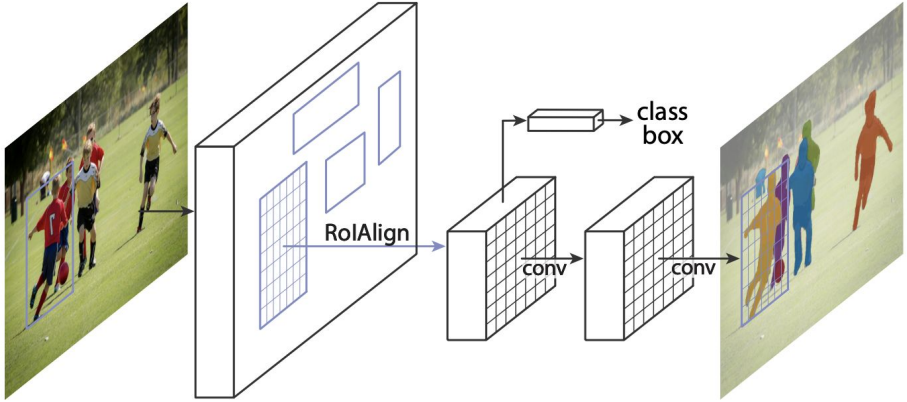


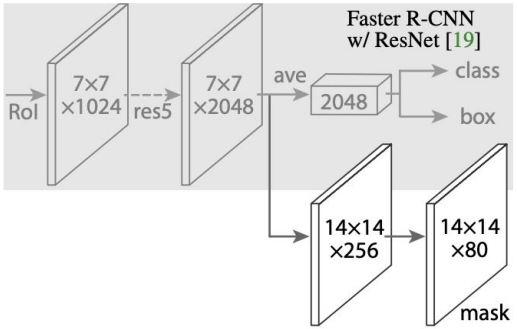
Fast/er R-CNN

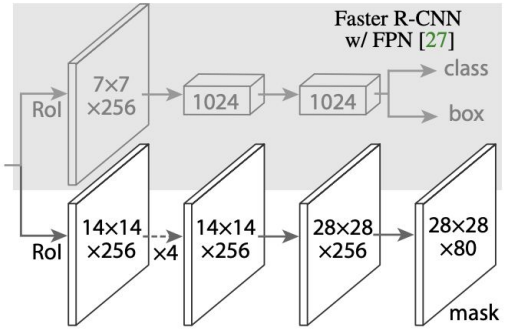


Mask R-CNN

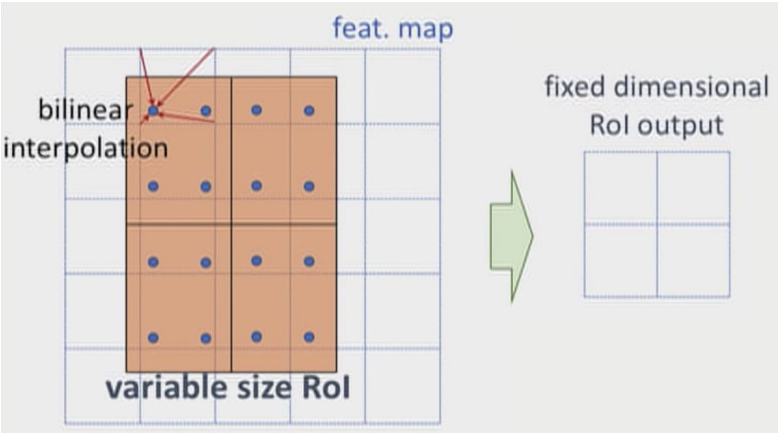
Mask R-CNN

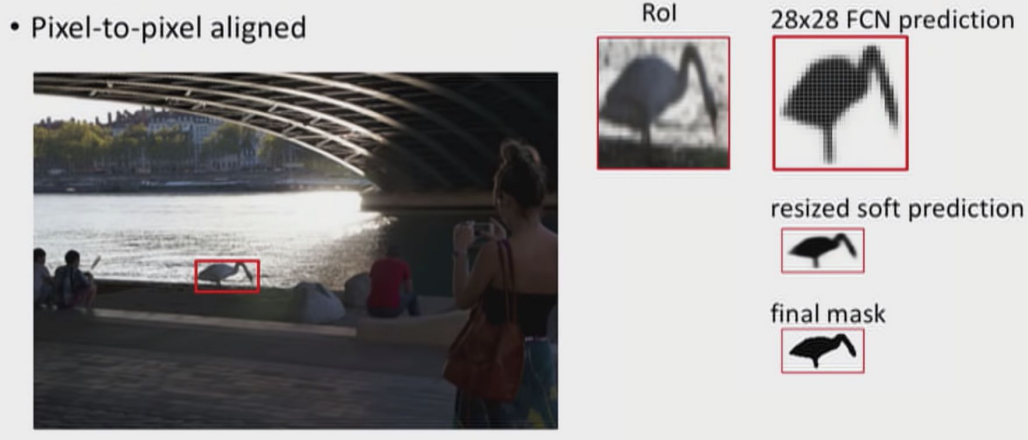

RoIAlign
class
box
conv
conv


Faster R-CNN
w/ ResNet [19]
RoI
7×7
×1024
res5
7×7
×2048
ave
2048
class
box
14×14
×256
14×14
×80
mask


Faster R-CNN
w/ FPN [27]
RoI
7×7
×256
1024
1024
class
box
RoI
14×14
×256
×4
14×14
×256
28×28
×256
28×28
×80
mask

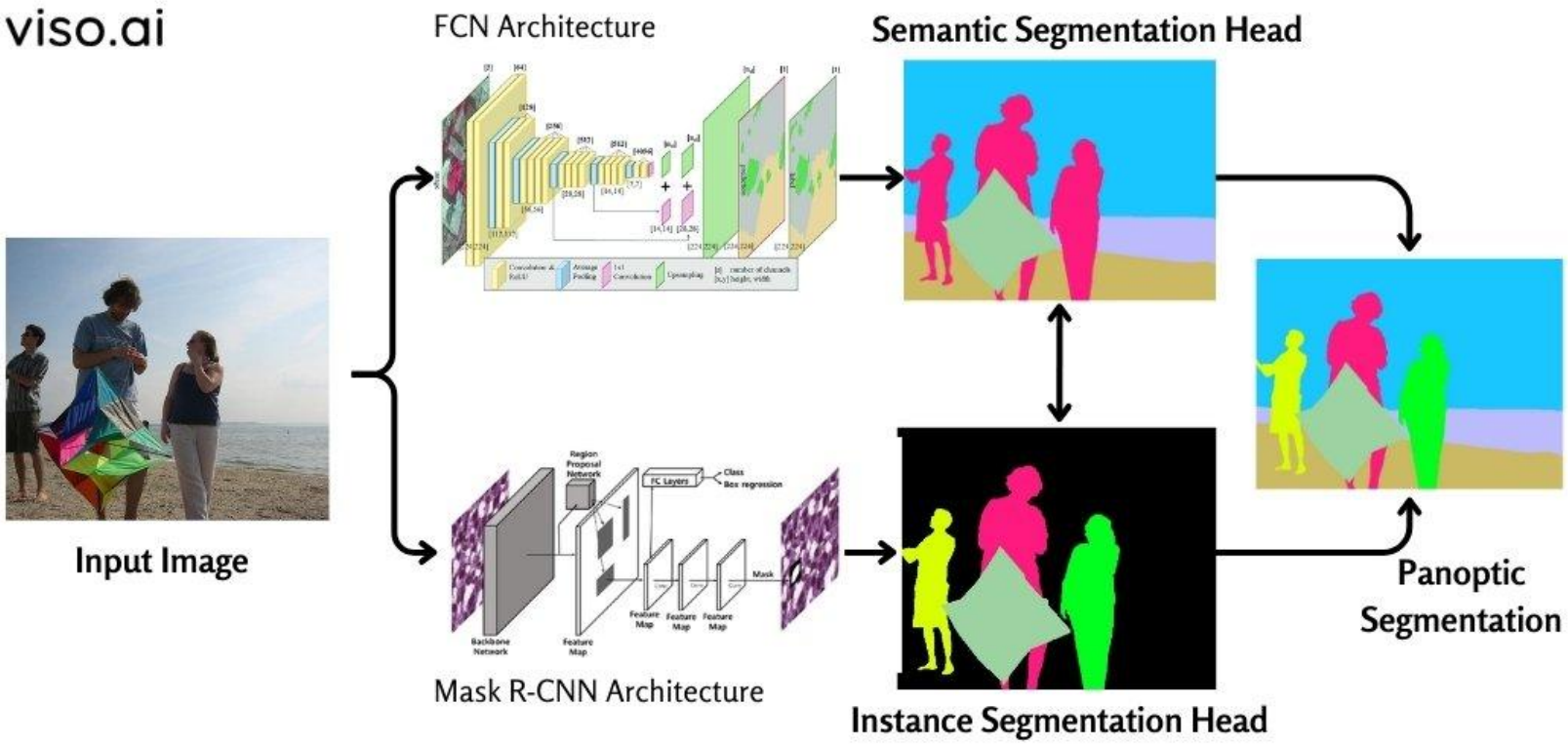
Mask R-CNN: RoI Align





Mask R-CNN: Results

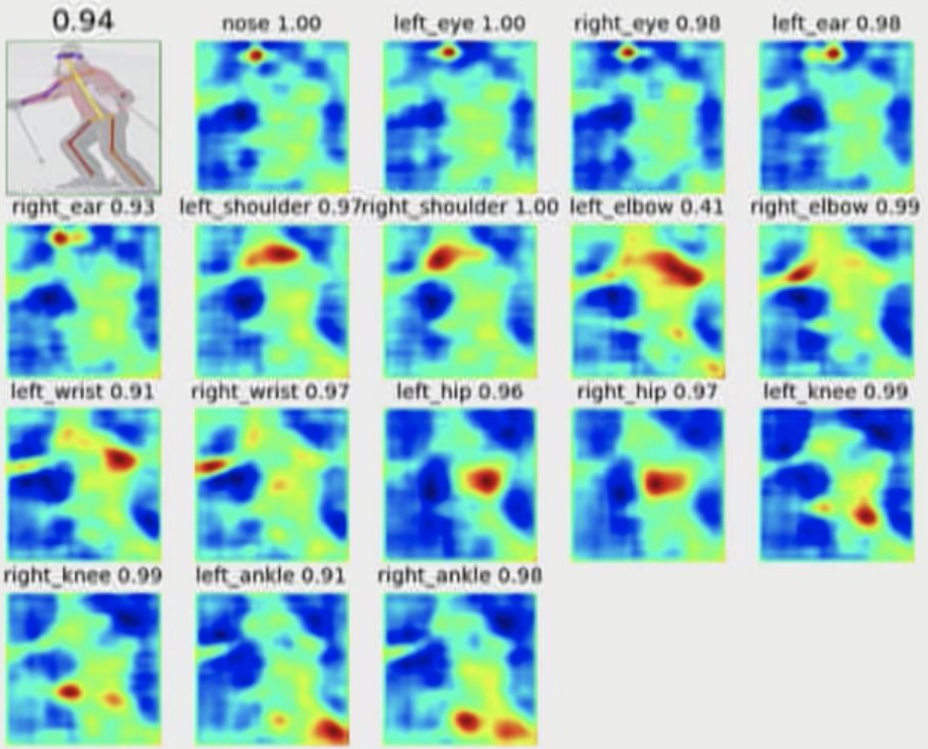
Panoptic segmentation with Mask R-CNN



Bonus: Pose estimation

- keypoint = 1-hot mask
- human pose = 17 masks

- One framework for
  - ✓bbox
  - ✓mask
  - ✓keypoint

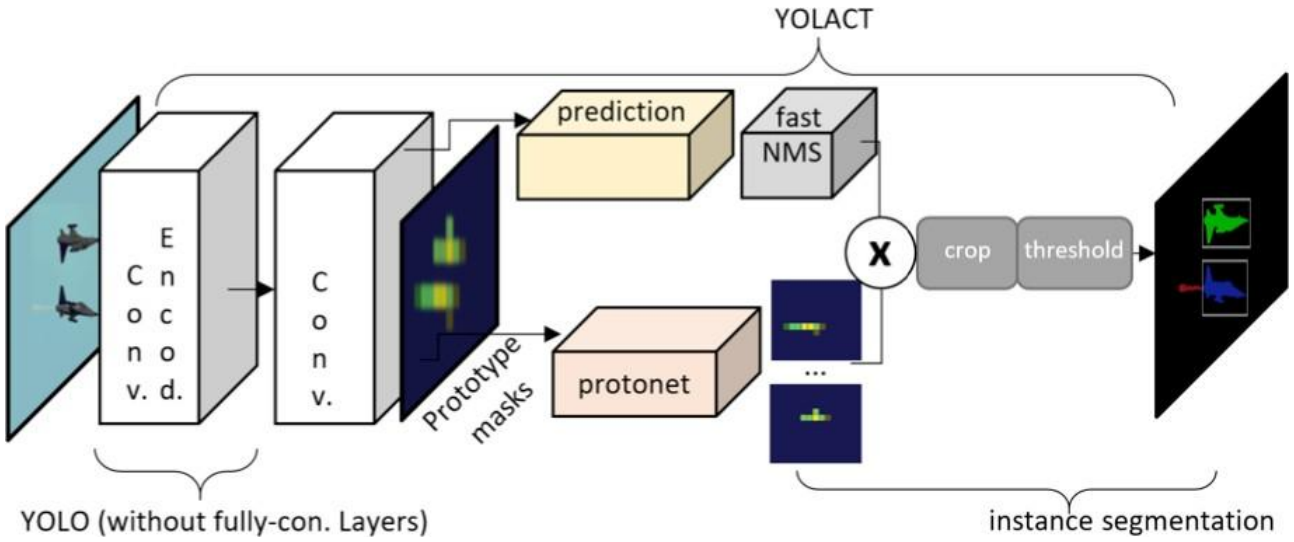


Bonus: Pose estimation

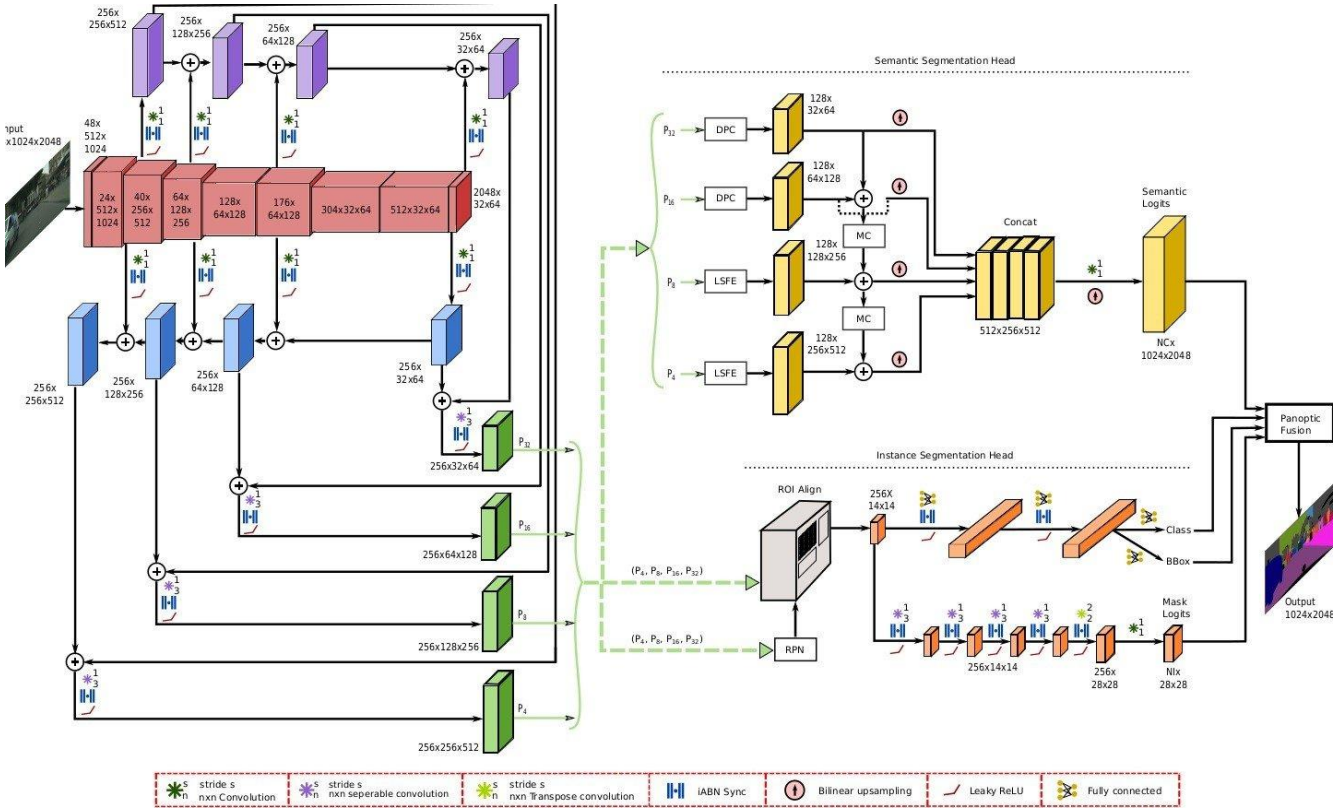
YOLACT

31.2% mAP on COCO dataset

~ 30 fps



EfficientPS



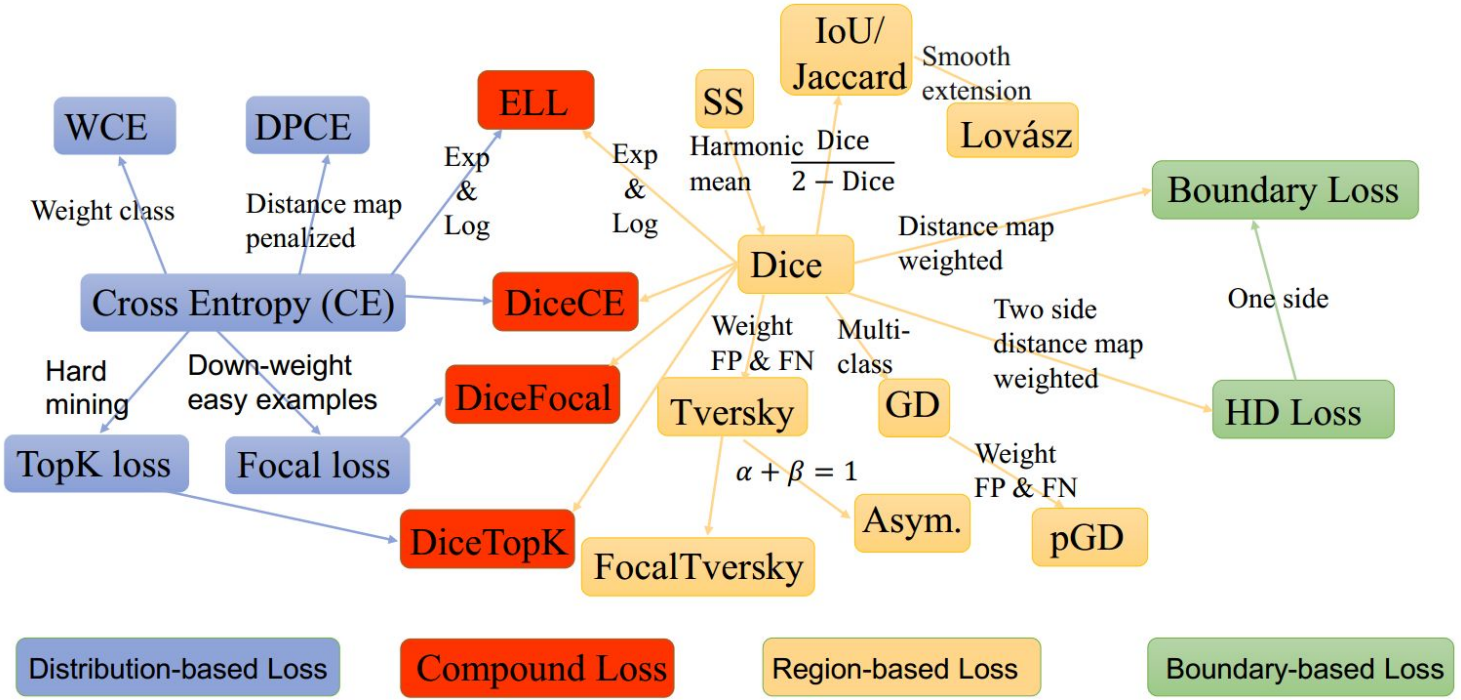
SAM

Segment Anything Model is universal solution for such problems

Further development
https://github.com/yangchris11/samurai

Loss functions

Segmentation Loss Odyssey



https://arxiv.org/pdf/2005.13449.pdf

Distribution-based Losses

- Crossentropy
- Focal loss
- Top-k loss
- etc

$$L_{CE} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c=1}^{C} g_i^c log s_i^c$$

$$L_{FL} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c}^{C}(1-s_i^c)^{\gamma} g_i^c log s_i^c$$

Region-based Losses

- Dice
- IoU
- Tversky loss
- Asymmetric similarity loss
- etc

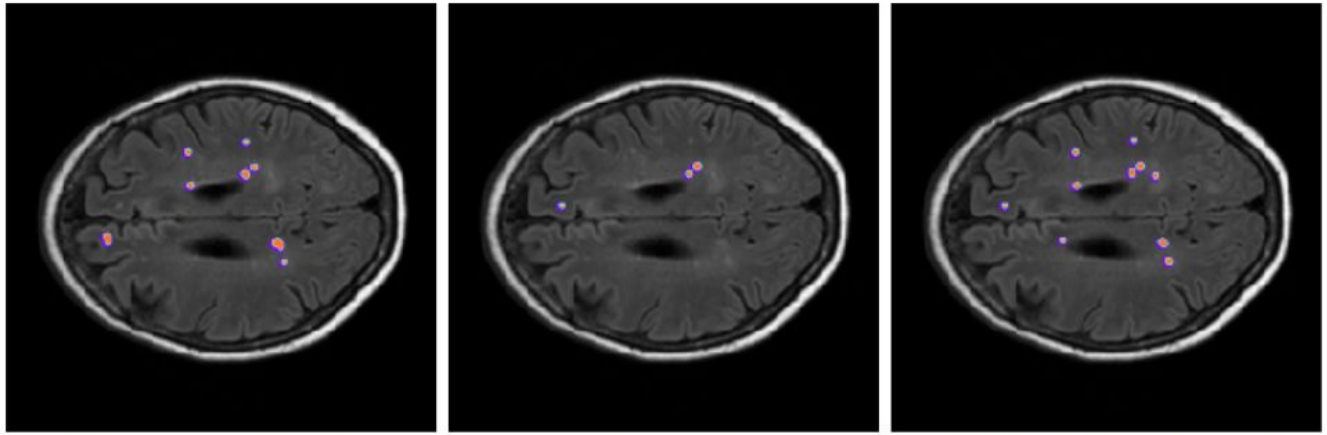
$$L_{Dice} = 1 - \frac{2\sum_{i=1}^{N}\sum_{c=1}^{C} g_i^c s_i^c}{\sum_{i=1}^{N}\sum_{c=1}^{C} g_i^{c2} + \sum_{i=1}^{N}\sum_{c=1}^{C} s_i^{c2}}$$

$$L_{IoU} = 1 - \frac{\sum_{i=1}^{N}\sum_{c=1}^{C} g_i^c s_i^c}{\sum_{i=1}^{N}\sum_{c=1}^{C} (g_i^c + s_i^c - g_i^c s_i^c)}$$

Boundary-based Losses

- Boundary (BD) loss
- Hausdorff Distance (HD) loss

Used for pixelwise quality segmentation



(a) Ground truth (b) GDL (c) GDL w/ boundary loss

- https://arxiv.org/pdf/2006.14822.pdf
- https://github.com/JunMa11/SegLossOdyssey
  - https://github.com/shruti-jadon/Semantic-Segmentation-Loss-Functions
- https://www.sciencedirect.com/science/article/abs/pii/S1361841521000815
- https://github.com/LIVIAETS/boundary-loss
- https://arxiv.org/pdf/1812.07032.pdf
- Transposed convolution visualization
- https://www.cvat.ai/
- https://github.com/hustvl/YOLOP/tree/main
-

Revise

- Segmentation task
- Simple solutions
- Upsampling methods
  - Unpooling
  - Transposed convolution
- FCN
- DeconvNet
- SegNet
- U-Net
- Mask R-CNN

Next time

- Variational AutoEncoders
- GANs

Interactive playground for image generation